



[Home](#) » [Get Started](#) » [Standard Setup of Toolchain for Windows](#) »

Establish Serial Connection with ESP32

[Edit on GitHub](#)

Note

There is a newer bugfix release of this version. The latest bugfix release is [v5.1.6](#)

Establish Serial Connection with ESP32

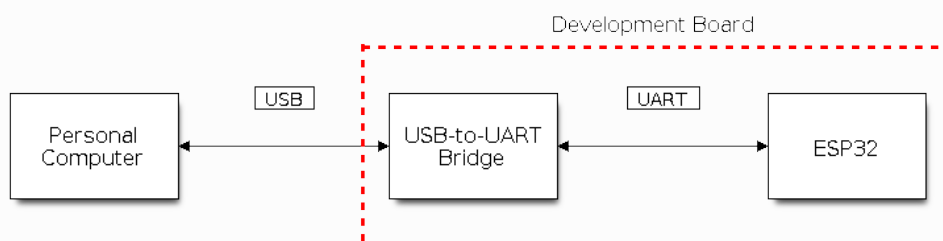
[\[中文\]](#)

Establishing a serial connection with the ESP32 target device could be done using a USB-to-UART bridge.

Some development boards have the USB-to-UART bridge installed. If a board does not have a bridge then an external bridge may be used.

USB-to-UART Bridge on Development Board

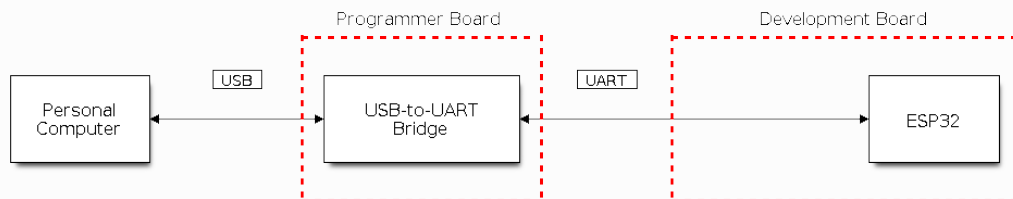
For boards with an installed USB-to-UART bridge, the connection between the personal computer and the bridge is USB and between the bridge and ESP32 is UART.



Development Board with USB-to-UART Bridge

External USB-to-UART Bridge

Sometimes the USB-to-UART bridge is external. This is often used in small development boards or finished products when space and costs are crucial.



External USB-to-UART Bridge

Flash using UART

This section provides guidance on how to establish a serial connection between ESP32 and PC using USB-to-UART Bridge, either installed on the development board or external.

Connect ESP32 to PC

Connect the ESP32 board to the PC using the USB cable. If device driver does not install automatically, identify USB-to-UART bridge on your ESP32 board (or external converter dongle), search for drivers in internet and install them.

Below is the list of USB to serial converter chips installed on most of the ESP32 boards produced by Espressif together with links to the drivers:

- CP210x: [CP210x USB to UART Bridge VCP Drivers](#)
- FTDI: [FTDI Virtual COM Port Drivers](#)

Please check the board user guide for specific USB-to-UART bridge chip used. The drivers above are primarily for reference. Under normal circumstances, the drivers should be bundled with an operating system and automatically installed upon connecting the board to the PC.

For devices downloaded using a USB-to-UART bridge, you can run the following command including the optional argument to define the baud rate.

```
idf.py -p PORT [-b BAUD] flash
```

You can change the flasher baud rate by replacing `BAUD` with the baud rate you need. The default baud rate is `460800`.

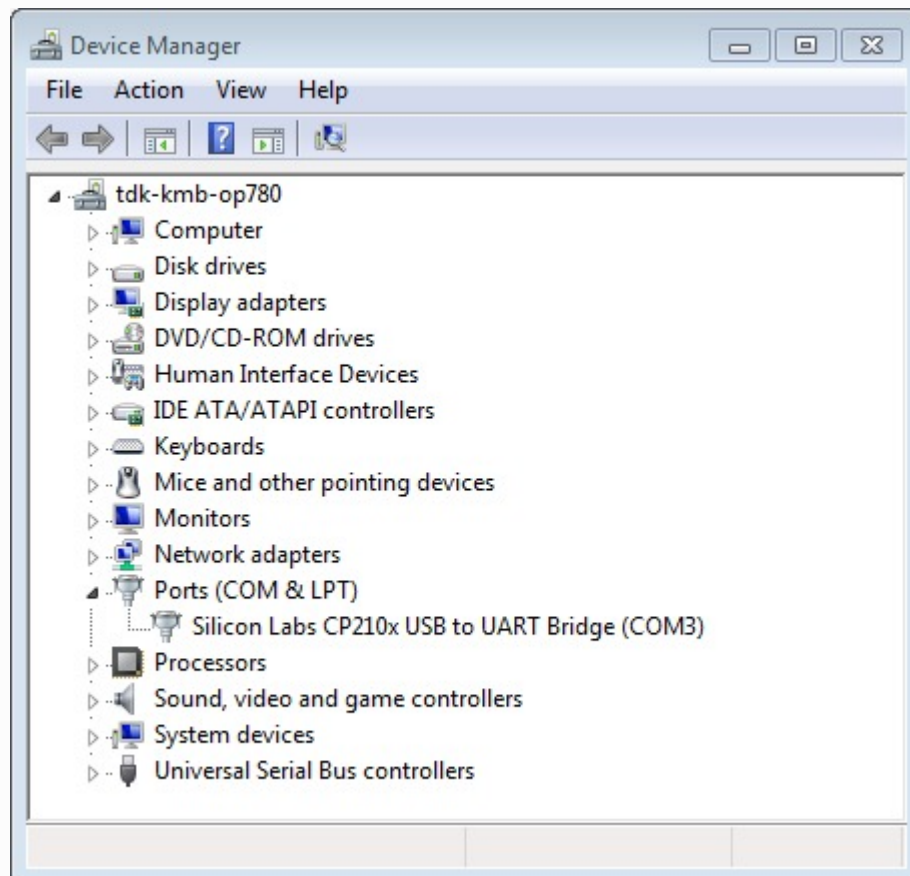
Note

If the device does not support the auto download mode, you need to get into the download mode manually. To do so, press and hold the **BOOT** button and then press the **RESET** button once. After that release the **BOOT** button.

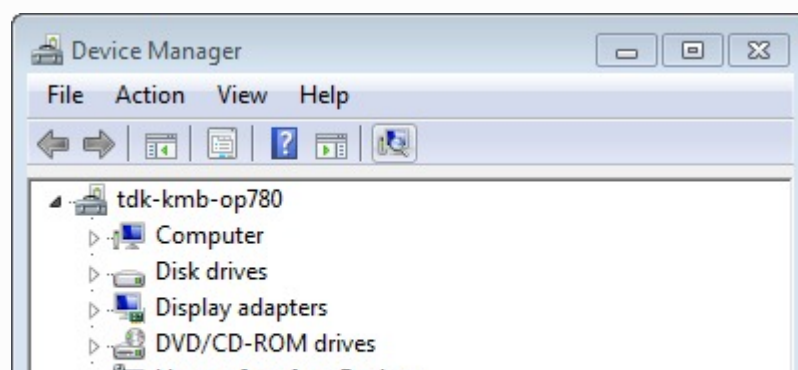
Check port on Windows

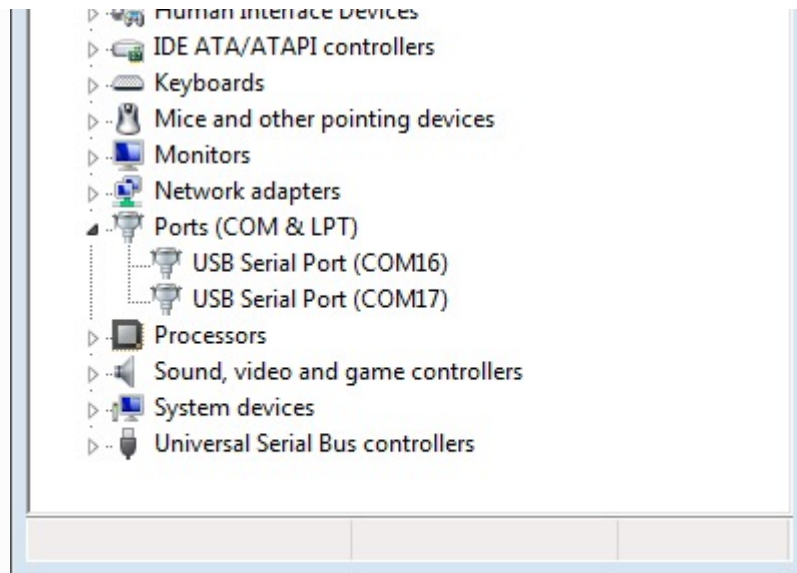
Check the list of identified COM ports in the Windows Device Manager. Disconnect ESP32 and connect it back, to verify which port disappears from the list and then shows back again.

Figures below show serial port for ESP32 DevKitC and ESP32 WROVER KIT



USB to UART bridge of ESP32-DevKitC in Windows Device Manager





Two USB Serial Ports of ESP-WROVER-KIT in Windows Device Manager

Check port on Linux and macOS

To check the device name for the serial port of your ESP32 board (or external converter dongle), run this command two times, first with the board / dongle unplugged, then with plugged in. The port which appears the second time is the one you need:

Linux

```
ls /dev/tty*
```

macOS

```
ls /dev/cu.*
```

Note

macOS users: if you don't see the serial port then check you have the USB/serial drivers installed. See Section [Connect ESP32 to PC](#) for links to drivers. For macOS High Sierra (10.13), you may also have to explicitly allow the drivers to load. Open System Preferences - > Security & Privacy -> General and check if there is a message shown here about "System Software from developer ..." where the developer name is Silicon Labs or FTDI.

Adding user to `dialout` on Linux

The currently logged user should have read and write access the serial port over USB. On most Linux distributions, this is done by adding the user to `dialout` group with the following

command:

```
sudo usermod -a -G dialout $USER
```

on Arch Linux this is done by adding the user to `uucp` group with the following command:

```
sudo usermod -a -G uucp $USER
```

Make sure you re-login to enable read and write permissions for the serial port.

Verify serial connection

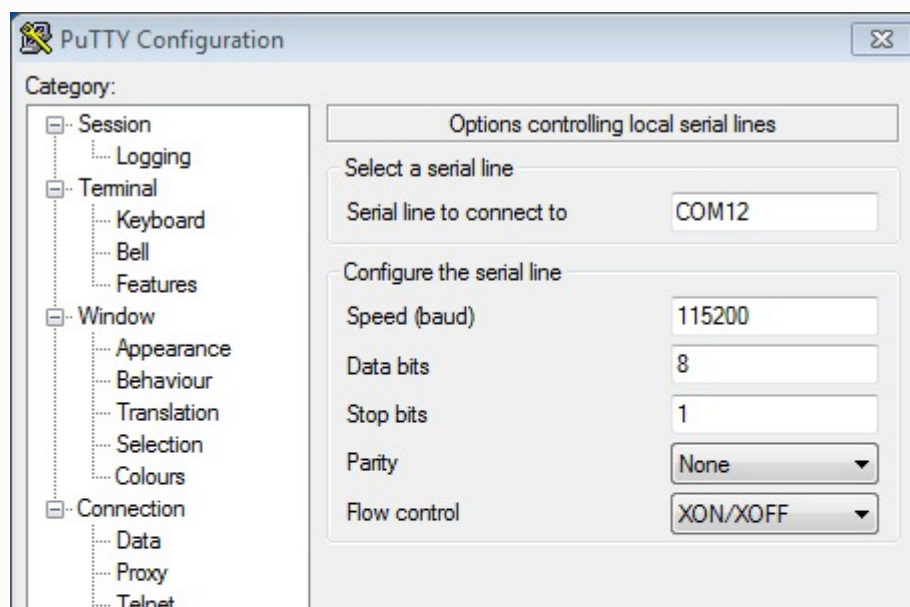
Now verify that the serial connection is operational. You can do this using a serial terminal program by checking if you get any output on the terminal after resetting ESP32.

The default console baud rate on ESP32 is 115200.

Windows and Linux

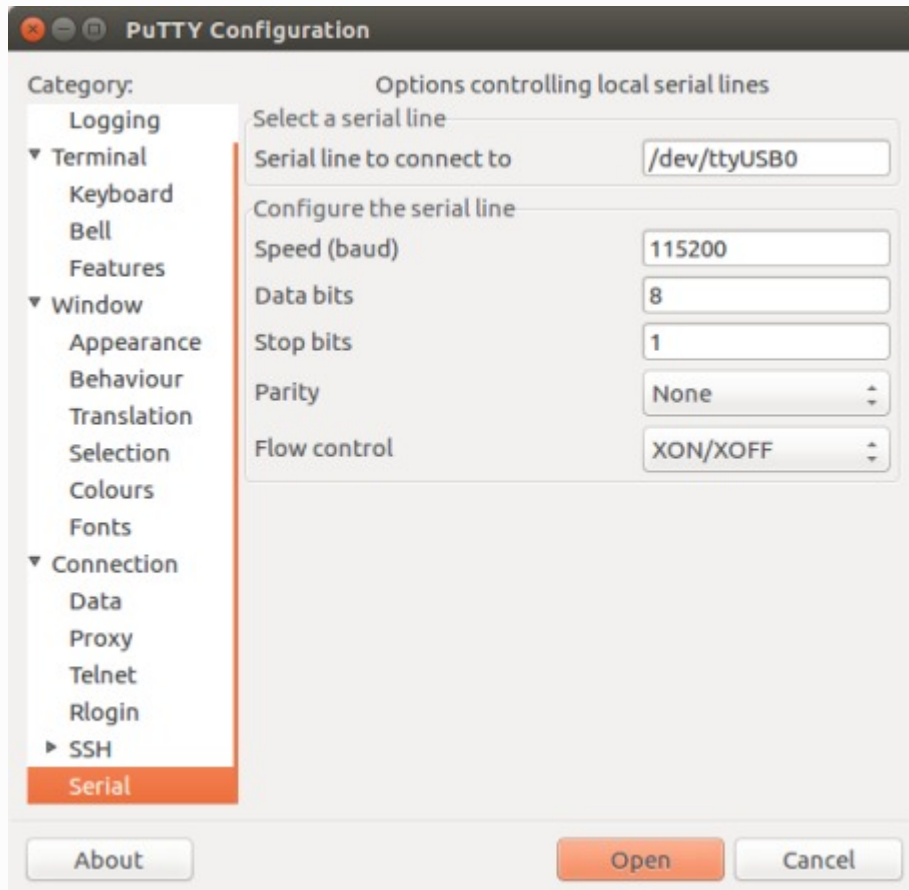
In this example we will use [PuTTY SSH Client](#) that is available for both Windows and Linux. You can use other serial programs and set communication parameters like below.

Run terminal and set identified serial port. Baud rate = 115200 (if needed, change this to the default baud rate of the chip in use), data bits = 8, stop bits = 1, and parity = N. Below are example screenshots of setting the port and such transmission parameters (in short described as 115200-8-1-N) on Windows and Linux. Remember to select exactly the same serial port you have identified in steps above.





Setting Serial Communication in PuTTY on Windows



Setting Serial Communication in PuTTY on Linux

Then open serial port in terminal and check, if you see any log printed out by ESP32. The log contents will depend on application loaded to ESP32, see [Example Output](#).

Note

Close the serial terminal after verification that communication is working. If you keep the terminal session open, the serial port will be inaccessible for uploading firmware later.

macOS

To spare you the trouble of installing a serial terminal program, macOS offers the **screen** command.

- As discussed in [Check port on Linux and macOS](#), run:

```
ls /dev/cu.*
```

- You should see similar output:

```
/dev/cu.Bluetooth-Incoming-Port /dev/cu.SLAB_USBtoUART /dev/cu.SLAB_USBtoUART7
```

- The output will vary depending on the type and the number of boards connected to your PC. Then pick the device name of your board and run (if needed, change “115200” to the default baud rate of the chip in use):

```
screen /dev/cu.device_name 115200
```

Replace `device_name` with the name found running `ls /dev/cu.*`.

- What you are looking for is some log displayed by the **screen**. The log contents will depend on application loaded to ESP32, see [Example Output](#). To exit the **screen** session type Ctrl-A + \.

Note

Do not forget to **exit the screen session** after verifying that the communication is working. If you fail to do it and just close the terminal window, the serial port will be inaccessible for uploading firmware later.

Example Output

An example log is shown below. Reset the board if you do not see anything.

```
ets Jun  8 2016 00:22:57

rst:0x5 (DEEPSLEEP_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)
ets Jun  8 2016 00:22:57

rst:0x7 (TG0WDT_SYS_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)
configsip: 0, SPIWP:0x00
clk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00
mode:DIO, clock div:2
load:0x3fff0008,len:8
load:0x3fff0010,len:3464
load:0x40078000,len:7828
load:0x40080000,len:252
entry 0x40080034
I (44) boot: ESP-IDF v2.0-rc1-401-gf9fba35 2nd stage bootloader
I (45) boot: compile time 18:48:10
```

If you can see readable log output, it means serial connection is working and you are ready to proceed with installation and finally upload an application to ESP32.

! Note

For some serial port wiring configurations, the serial RTS & DTR pins need to be disabled in the terminal program before the ESP32 will boot and produce serial output. This depends on the hardware itself, most development boards (including all Espressif boards) *do not* have this issue. The issue is present if RTS & DTR are wired directly to the EN & GPIO0 pins. See the [esptool documentation](#) for more details.

If you got here from [Step 5. First Steps on ESP-IDF](#) when installing s/w for ESP32 development, then you can continue with [Step 5. First Steps on ESP-IDF](#).

[Provide feedback about this document](#)

[← Previous](#)

[Next →](#)

© Copyright 2016 - 2023, Espressif Systems (Shanghai) Co., Ltd.

Built with [Sphinx](#) using a [theme](#) based on [Read the Docs Sphinx Theme](#).

[Download PDF](#)

**Note**

There is a newer bugfix release of this version. The latest bugfix release is [v5.1.6](#)

FreeRTOS (Overview)

Overview

FreeRTOS is an open source real-time operating system kernel that acts as the operating system for ESP-IDF applications and is integrated into ESP-IDF as a component. The FreeRTOS component in ESP-IDF contains ports of the FreeRTOS kernel for all the CPU architectures used by ESP targets (i.e., Xtensa and RISC-V). Furthermore, ESP-IDF provides different implementations of FreeRTOS in order to support SMP (Symmetric Multiprocessing) on multi-core ESP targets. This document provides an overview of the FreeRTOS component, the FreeRTOS implementations offered by ESP-IDF, and the common aspects across all implementations.

Implementations

The [official FreeRTOS](#) (henceforth referred to as Vanilla FreeRTOS) is a single-core RTOS. In order to support the various multi-core ESP targets, ESP-IDF supports different FreeRTOS implementations, namely **ESP-IDF FreeRTOS** and **Amazon SMP FreeRTOS**.

ESP-IDF FreeRTOS

ESP-IDF FreeRTOS is a FreeRTOS implementation based on Vanilla FreeRTOS v10.4.3, but contains significant modifications to support SMP. ESP-IDF FreeRTOS only supports two cores at most (i.e., dual core SMP), but is more optimized for this scenario by design. For more details regarding ESP-IDF FreeRTOS and its modifications, please refer to the [FreeRTOS \(ESP-IDF\)](#) document.

Note

ESP-IDF FreeRTOS is currently the default FreeRTOS implementation for ESP-IDF.

Amazon SMP FreeRTOS

Amazon SMP FreeRTOS is an SMP implementation of FreeRTOS that is officially supported by Amazon. Amazon SMP FreeRTOS is able to support N-cores (i.e., more than two cores). Amazon SMP FreeRTOS can be enabled via the [CONFIG_FREERTOS_SMP](#) option. For more details regarding Amazon SMP FreeRTOS, please refer to the [official Amazon SMP FreeRTOS documentation](#).

⚠ Warning

The Amazon SMP FreeRTOS implementation (and its port in ESP-IDF) are currently in experimental/beta state. Therefore, significant behavioral changes and breaking API changes can occur.

Configuration

Kernel Configuration

Vanilla FreeRTOS requires that ports and applications configure the kernel by adding various `#define config...` macros to `FreeRTOSConfig.h`. Vanilla FreeRTOS supports a list of kernel configuration options which allow various kernel behaviors and features to be enabled or disabled.

However, for all FreeRTOS ports in ESP-IDF, the `FreeRTOSConfig.h` file is considered **private and must not be modified by users**. A large number of kernel configuration options in `FreeRTOSConfig.h` are hard coded as they are either required or not supported in ESP-IDF. All kernel configuration options that are configurable by the user will be exposed via menuconfig under `Component Config/FreeRTOS/Kernel`.

For the full list of user configurable kernel options, see [Project Configuration](#). The list below highlights some commonly used kernel configuration options:

- [CONFIG_FREERTOS_UNICORE](#) will run FreeRTOS only on CPU0. Note that this is **not equivalent to running Vanilla FreeRTOS**. Furthermore, this option may affect behavior of components other than `freertos`. For more details regarding the effects of running FreeRTOS on a single core, refer to [ESP-IDF FreeRTOS Single Core](#) (if using ESP-IDF FreeRTOS) or the official Amazon SMP FreeRTOS documentation. Alternatively, users can also search for occurrences of `CONFIG_FREERTOS_UNICORE` in the ESP-IDF components.
- [CONFIG_FREERTOS_ENABLE_BACKWARD_COMPATIBILITY](#) enables backward

compatibility with some FreeRTOS macros/types/functions that were deprecated from v8.0 onwards.

Port Configuration

All other FreeRTOS related configuration options that are not part of the kernel configuration are exposed via menuconfig under `Component Config/FreeRTOS/Port`. These options configure aspects such as:

- The FreeRTOS ports themselves (e.g., tick timer selection, ISR stack size)
- Additional features added to the FreeRTOS implementation or ports

Using FreeRTOS

Application Entry Point

Unlike Vanilla FreeRTOS, users of FreeRTOS in ESP-IDF **must never call** `vTaskStartScheduler()` and `vTaskEndScheduler()`. Instead, ESP-IDF will start FreeRTOS automatically. Users must define a `void app_main(void)` function which acts as the entry point for user's application and is automatically called on ESP-IDF startup.

- Typically, users would spawn the rest of their application's task from `app_main`.
- The `app_main` function is allowed to return at any point (i.e., before the application terminates).
- The `app_main` function is called from the `main` task.

Background Tasks

During startup, ESP-IDF and FreeRTOS will automatically create multiple tasks that run in the background (listed in the the table below).

Task Name	Description
Idle Tasks (<code>IDLEx</code>)	An idle task (<code>IDLEx</code>) is created for (and pinned to) each CPU, where <code>x</code>
FreeRTOS Timer Task (<code>TmrSvc</code>)	FreeRTOS will create the Timer Service/Daemon Task if any FreeRTOS
Main Task (<code>main</code>)	Task that simply calls <code>app_main</code> . This task will self delete when <code>app_main</code>
IPC Tasks (<code>ipcx</code>)	When <code>CONFIG_FREERTOS_UNICORE</code> is false, an IPC task (<code>ipcx</code>) is cre
ESP Timer Task (<code>esp_timer</code>)	ESP-IDF will create the ESP Timer Task used to process ESP Timer callb.

Task Name	Description

Note

Note that if an application uses other ESP-IDF features (e.g., WiFi or Bluetooth), those features may create their own background tasks in addition to the tasks listed in the table above.

FreeRTOS Additions

ESP-IDF provides some supplemental features to FreeRTOS such as Ring Buffers, ESP-IDF style Tick and Idle Hooks, and TLSP deletion callbacks. See [FreeRTOS \(Supplemental Features\)](#) for more details.

FreeRTOS Heap

Vanilla FreeRTOS provides its own [selection of heap implementations](#). However, ESP-IDF already implements its own heap (see [Heap Memory Allocation](#)), thus ESP-IDF does not make use of the heap implementations provided by Vanilla FreeRTOS. All FreeRTOS ports in ESP-IDF map FreeRTOS memory allocation/free calls (e.g., `pvPortMalloc()` and `pvPortFree()`) to ESP-IDF heap API (i.e., `heap_caps_malloc()` and `heap_caps_free()`). However, the FreeRTOS ports ensure that all dynamic memory allocated by FreeRTOS is placed in internal memory.

Note

If users wish to place FreeRTOS tasks/objects in external memory, users can use the following methods:

- Allocate the task/object using one of the `...CreateWithCaps()` API such as `xTaskCreateWithCaps()` and `xQueueCreateWithCaps()` (see [IDF Additional API](#) for more details).
- Manually allocate external memory for those objects using `heap_caps_malloc()`, then create the objects from the allocated memory using one of the `...CreateStatic()` FreeRTOS functions.

[Provide feedback about this document](#)

© Copyright 2016 - 2023, Espressif Systems (Shanghai) Co., Ltd.

Built with [Sphinx](#) using a [theme](#) based on [Read the Docs Sphinx Theme](#).

 [Download PDF](#)

**Note**

There is a newer bugfix release of this version. The latest bugfix release is [v5.1.6](#)

Universal Asynchronous Receiver/Transmitter (UART)

Introduction

A Universal Asynchronous Receiver/Transmitter (UART) is a hardware feature that handles communication (i.e., timing requirements and data framing) using widely-adopted asynchronous serial communication interfaces, such as RS232, RS422, and RS485. A UART provides a widely adopted and cheap method to realize full-duplex or half-duplex data exchange among different devices.

The ESP32 chip has 3 UART controllers (also referred to as port), each featuring an identical set of registers to simplify programming and for more flexibility.

Each UART controller is independently configurable with parameters such as baud rate, data bit length, bit ordering, number of stop bits, parity bit, etc. All the controllers are compatible with UART-enabled devices from various manufacturers and can also support Infrared Data Association (IrDA) protocols.

Functional Overview

The overview describes how to establish communication between an ESP32 and other UART devices using the functions and data types of the UART driver. A typical programming workflow is broken down into the sections provided below:

1. [Set Communication Parameters](#) - Setting baud rate, data bits, stop bits, etc.
2. [Set Communication Pins](#) - Assigning pins for connection to a device
3. [Install Drivers](#) - Allocating ESP32's resources for the UART driver

4. [Run UART Communication](#) - Sending/receiving data
5. [Use Interrupts](#) - Triggering interrupts on specific communication events
6. [Deleting a Driver](#) - Freeing allocated resources if a UART communication is no longer required

Steps 1 to 3 comprise the configuration stage. Step 4 is where the UART starts operating. Steps 5 and 6 are optional.

The UART driver's functions identify each of the UART controllers using `uart_port_t`. This identification is needed for all the following function calls.

Set Communication Parameters

UART communication parameters can be configured all in a single step or individually in multiple steps.

Single Step

Call the function `uart_param_config()` and pass to it a `uart_config_t` structure. The `uart_config_t` structure should contain all the required parameters. See the example below.

```
const uart_port_t uart_num = UART_NUM_2;
uart_config_t uart_config = {
    .baud_rate = 115200,
    .data_bits = UART_DATA_8_BITS,
    .parity = UART_PARITY_DISABLE,
    .stop_bits = UART_STOP_BITS_1,
    .flow_ctrl = UART_HW_FLOWCTRL_CTS_RTS,
    .rx_flow_ctrl_thresh = 122,
};
// Configure UART parameters
ESP_ERROR_CHECK(uart_param_config(uart_num, &uart_config));
```

For more information on how to configure the hardware flow control options, please refer to [peripherals/uart/uart_echo](#).

Multiple Steps

Configure specific parameters individually by calling a dedicated function from the table given below. These functions are also useful if re-configuring a single parameter.

Functions for Configuring specific parameters individually

Parameter to Configure	Function
Baud rate	<code>uart_set_baudrate()</code>
Number of transmitted bits	<code>uart_set_word_length()</code> selected out of <code>uart_word_length_t</code>
Parity control	<code>uart_set_parity()</code> selected out of <code>uart_parity_t</code>
Number of stop bits	<code>uart_set_stop_bits()</code> selected out of <code>uart_stop_bits_t</code>
Hardware flow control mode	<code>uart_set_hw_flow_ctrl()</code> selected out of <code>uart_hw_flowcontrol_t</code>
Communication mode	<code>uart_set_mode()</code> selected out of <code>uart_mode_t</code>

Each of the above functions has a `_get_` counterpart to check the currently set value. For example, to check the current baud rate value, call `uart_get_baudrate()`.

Set Communication Pins

After setting communication parameters, configure the physical GPIO pins to which the other UART device will be connected. For this, call the function `uart_set_pin()` and specify the GPIO pin numbers to which the driver should route the Tx, Rx, RTS, and CTS signals. If you want to keep a currently allocated pin number for a specific signal, pass the macro `UART_PIN_NO_CHANGE`.

The same macro `UART_PIN_NO_CHANGE` should be specified for pins that will not be used.

```
// Set UART pins(TX: IO4, RX: IO5, RTS: IO18, CTS: IO19)
ESP_ERROR_CHECK(uart_set_pin(UART_NUM_2, 4, 5, 18, 19));
```

Install Drivers

Once the communication pins are set, install the driver by calling `uart_driver_install()` and specify the following parameters:

- Size of Tx ring buffer
- Size of Rx ring buffer
- Event queue handle and size
- Flags to allocate an interrupt

The function will allocate the required internal resources for the UART driver.

```
// Setup UART buffered IO with event queue
const int uart_buffer_size = (1024 * 2);
```



```
QueueHandle_t uart_queue;
// Install UART driver using an event queue here
ESP_ERROR_CHECK(uart_driver_install(UART_NUM_2, uart_buffer_size, \
                                   uart_buffer_size, 10, &uart_queue, 0));
```

Once this step is complete, you can connect the external UART device and check the communication.

Run UART Communication

Serial communication is controlled by each UART controller's finite state machine (FSM).

The process of sending data involves the following steps:

1. Write data into Tx FIFO buffer
2. FSM serializes the data
3. FSM sends the data out

The process of receiving data is similar, but the steps are reversed:

1. FSM processes an incoming serial stream and parallelizes it
2. FSM writes the data into Rx FIFO buffer
3. Read the data from Rx FIFO buffer

Therefore, an application will only write and read data from a specific buffer using `uart_write_bytes()` and `uart_read_bytes()` respectively, and the FSM will do the rest.

Transmit Data

After preparing the data for transmission, call the function `uart_write_bytes()` and pass the data buffer's address and data length to it. The function will copy the data to the Tx ring buffer (either immediately or after enough space is available), and then exit. When there is free space in the Tx FIFO buffer, an interrupt service routine (ISR) moves the data from the Tx ring buffer to the Tx FIFO buffer in the background. The code below demonstrates the use of this function.

```
// Write data to UART.
char* test_str = "This is a test string.\n";
uart_write_bytes(uart_num, (const char*)test_str, strlen(test_str));
```

The function `uart_write_bytes_with_break()` is similar to `uart_write_bytes()` but adds a serial break signal at the end of the transmission. A 'serial break signal' means holding the Tx line low for a period longer than one data frame.

```
// Write data to UART, end with a break signal.
uart_write_bytes_with_break(uart_num, "test break\n", strlen("test break\n"), 100);
```

Another function for writing data to the Tx FIFO buffer is `uart_tx_chars()`. Unlike `uart_write_bytes()`, this function will not block until space is available. Instead, it will write all data which can immediately fit into the hardware Tx FIFO, and then return the number of bytes that were written.

There is a 'companion' function `uart_wait_tx_done()` that monitors the status of the Tx FIFO buffer and returns once it is empty.

```
// Wait for packet to be sent
const uart_port_t uart_num = UART_NUM_2;
ESP_ERROR_CHECK(uart_wait_tx_done(uart_num, 100)); // wait timeout is 100 RTOS ticks (TickType_t)
```

Receive Data

Once the data is received by the UART and saved in the Rx FIFO buffer, it needs to be retrieved using the function `uart_read_bytes()`. Before reading data, you can check the number of bytes available in the Rx FIFO buffer by calling `uart_get_buffered_data_len()`. An example of using these functions is given below.

```
// Read data from UART.
const uart_port_t uart_num = UART_NUM_2;
uint8_t data[128];
int length = 0;
ESP_ERROR_CHECK(uart_get_buffered_data_len(uart_num, (size_t*)&length));
length = uart_read_bytes(uart_num, data, length, 100);
```

If the data in the Rx FIFO buffer is no longer needed, you can clear the buffer by calling `uart_flush()`.

Software Flow Control

If the hardware flow control is disabled, you can manually set the RTS and DTR signal levels by using the functions `uart_set_rts()` and `uart_set_dtr()` respectively.

Communication Mode Selection

The UART controller supports a number of communication modes. A mode can be selected using the function `uart_set_mode()`. Once a specific mode is selected, the UART driver will

handle the behavior of a connected UART device accordingly. As an example, it can control the RS485 driver chip using the RTS line to allow half-duplex RS485 communication.

```
// Setup UART in rs485 half duplex mode
ESP_ERROR_CHECK(uart_set_mode(uart_num, UART_MODE_RS485_HALF_DUPLEX));
```

Use Interrupts

There are many interrupts that can be generated depending on specific UART states or detected errors. The full list of available interrupts is provided in *ESP32 Technical Reference Manual > UART Controller (UART) > UART Interrupts and UHCI Interrupts* [PDF]. You can enable or disable specific interrupts by calling `uart_enable_intr_mask()` OR `uart_disable_intr_mask()` respectively.

The `uart_driver_install()` function installs the driver's internal interrupt handler to manage the Tx and Rx ring buffers and provides high-level API functions like events (see below).

The API provides a convenient way to handle specific interrupts discussed in this document by wrapping them into dedicated functions:

- **Event detection:** There are several events defined in `uart_event_type_t` that may be reported to a user application using the FreeRTOS queue functionality. You can enable this functionality when calling `uart_driver_install()` described in [Install Drivers](#). An example of using Event detection can be found in [peripherals/uart/uart_events](#).
- **FIFO space threshold or transmission timeout reached:** The Tx and Rx FIFO buffers can trigger an interrupt when they are filled with a specific number of characters, or on a timeout of sending or receiving data. To use these interrupts, do the following:
 - Configure respective threshold values of the buffer length and timeout by entering them in the structure `uart_intr_config_t` and calling `uart_intr_config()`
 - Enable the interrupts using the functions `uart_enable_tx_intr()` and `uart_enable_rx_intr()`
 - Disable these interrupts using the corresponding functions `uart_disable_tx_intr()` OR `uart_disable_rx_intr()`
- **Pattern detection:** An interrupt triggered on detecting a 'pattern' of the same character being received/sent repeatedly. This functionality is demonstrated in the example [peripherals/uart/uart_events](#). It can be used, e.g., to detect a command string with a specific number of identical characters (the 'pattern') at the end. The following functions are available:
 - Configure and enable this interrupt using `uart_enable_pattern_det_baud_intr()`
 - Disable the interrupt using `uart_disable_pattern_det_intr()`

Macros

The API also defines several macros. For example, `UART_FIFO_LEN` defines the length of hardware FIFO buffers; `UART_BITRATE_MAX` gives the maximum baud rate supported by the UART controllers, etc.

Deleting a Driver

If the communication established with `uart_driver_install()` is no longer required, the driver can be removed to free allocated resources by calling `uart_driver_delete()`.

Overview of RS485 Specific Communication Options

Note

The following section will use `[UART_REGISTER_NAME].[UART_FIELD_BIT]` to refer to UART register fields/bits. For more information on a specific option bit, see *ESP32 Technical Reference Manual > UART Controller (UART) > Register Summary* [PDF]. Use the register name to navigate to the register description and then find the field/bit.

- `UART_RS485_CONF_REG.UART_RS485_EN`: setting this bit enables RS485 communication mode support.
- `UART_RS485_CONF_REG.UART_RS485TX_RX_EN`: if this bit is set, the transmitter's output signal loops back to the receiver's input signal.
- `UART_RS485_CONF_REG.UART_RS485RXBY_TX_EN`: if this bit is set, the transmitter will still be sending data if the receiver is busy (remove collisions automatically by hardware).

The ESP32's RS485 UART hardware can detect signal collisions during transmission of a datagram and generate the interrupt `UART_RS485_CLASH_INT` if this interrupt is enabled. The term collision means that a transmitted datagram is not equal to the one received on the other end. Data collisions are usually associated with the presence of other active devices on the bus or might occur due to bus errors.

The collision detection feature allows handling collisions when their interrupts are activated and triggered. The interrupts `UART_RS485_FRM_ERR_INT` and `UART_RS485_PARITY_ERR_INT` can be used with the collision detection feature to control frame errors and parity bit errors accordingly in RS485 mode. This functionality is supported in the UART driver and can be used by selecting the `UART_MODE_RS485_APP_CTRL` mode (see the function `uart_set_mode()`).

The collision detection feature can work with circuit A and circuit C (see Section [Interface](#)

Connection Options). In the case of using circuit A or B, the RTS pin connected to the DE pin of the bus driver should be controlled by the user application. Use the function

`uart_get_collision_flag()` to check if the collision detection flag has been raised.

The ESP32 UART controllers themselves do not support half-duplex communication as they cannot provide automatic control of the RTS pin connected to the RE/DE input of RS485 bus driver. However, half-duplex communication can be achieved via software control of the RTS pin by the UART driver. This can be enabled by selecting the `UART_MODE_RS485_HALF_DUPLEX` mode when calling `uart_set_mode()`.

Once the host starts writing data to the Tx FIFO buffer, the UART driver automatically asserts the RTS pin (logic 1); once the last bit of the data has been transmitted, the driver de-asserts the RTS pin (logic 0). To use this mode, the software would have to disable the hardware flow control function. This mode works with all the used circuits shown below.

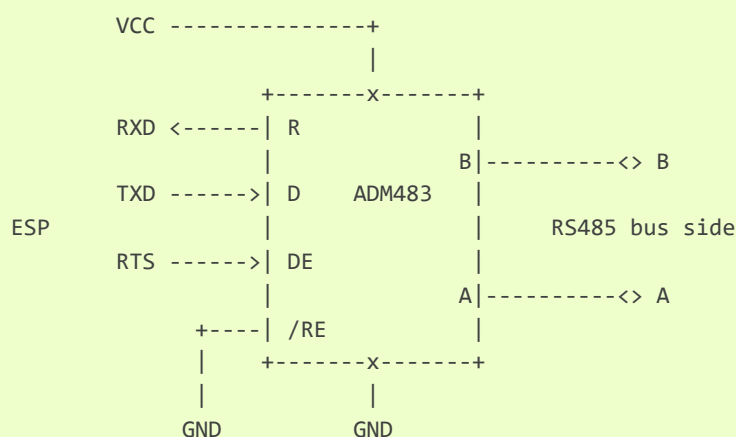
Interface Connection Options

This section provides example schematics to demonstrate the basic aspects of ESP32's RS485 interface connection.

Note

- The schematics below do **not** necessarily contain **all required elements**.
- The **analog devices** ADM483 & ADM2483 are examples of common RS485 transceivers and **can be replaced** with other similar transceivers.

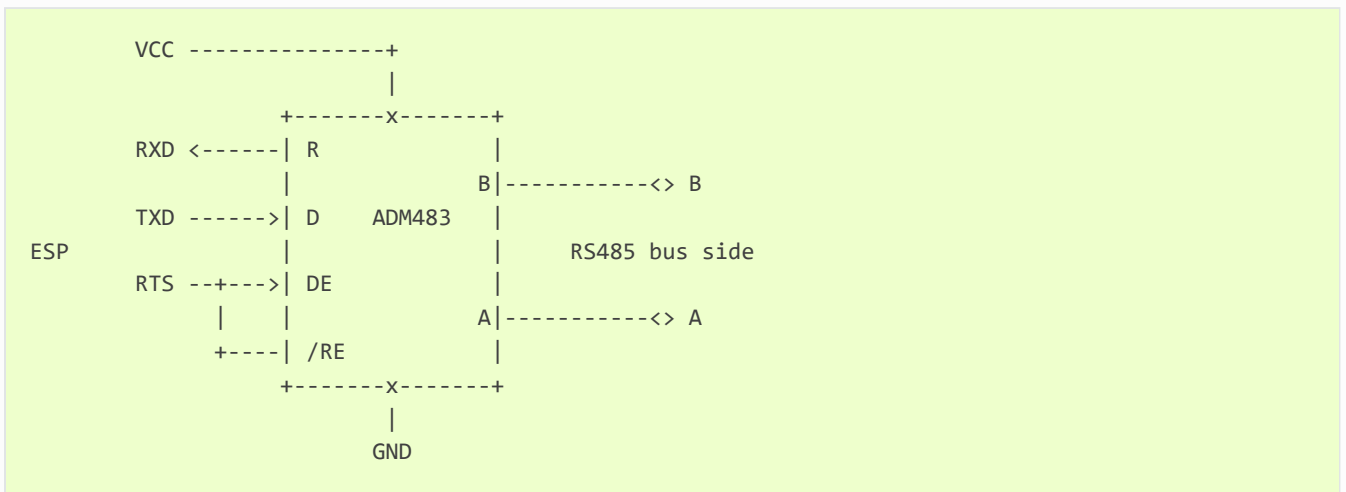
Circuit A: Collision Detection Circuit



This circuit is preferable because it allows for collision detection and is quite simple at the same time. The receiver in the line driver is constantly enabled, which allows the UART to monitor the RS485 bus. Echo suppression is performed by the UART peripheral when the bit

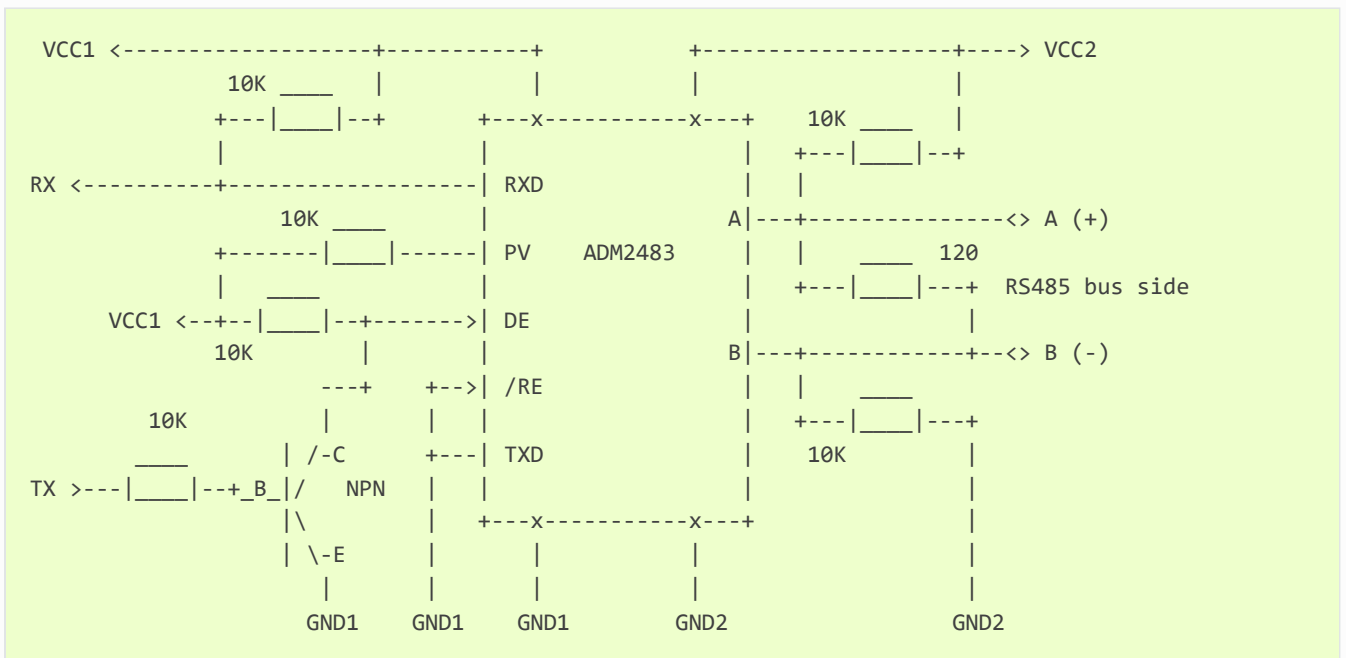
UART_RS485_CONF_REG.UART_RS485TX_RX_EN is enabled.

Circuit B: Manual Switching Transmitter/Receiver Without Collision Detection



This circuit does not allow for collision detection. It suppresses the null bytes that the hardware receives when the bit `UART_RS485_CONF_REG.UART_RS485TX_RX_EN` is set. The bit `UART_RS485_CONF_REG.UART_RS485RXBY_TX_EN` is not applicable in this case.

Circuit C: Auto Switching Transmitter/Receiver



This galvanically isolated circuit does not require RTS pin control by a software application or driver because it controls the transceiver direction automatically. However, it requires suppressing null bytes during transmission by setting `UART_RS485_CONF_REG.UART_RS485RXBY_TX_EN` to 1 and `UART_RS485_CONF_REG.UART_RS485TX_RX_EN` to 0. This setup can work in any RS485 UART mode or even in `UART_MODE_UART`.

Application Examples

The table below describes the code examples available in the directory [peripherals/uart/](#).

Code Example	Description
peripherals/uart/uart_echo	Configuring UART settings, installing the UART driver, and re
peripherals/uart/uart_events	Reporting various communication events, using pattern dete
peripherals/uart/uart_async_rxtxtasks	Transmitting and receiving data in two separate FreeRTOS ta
peripherals/uart/uart_select	Using synchronous I/O multiplexing for UART file descriptor
peripherals/uart/uart_echo_rs485	Setting up UART driver to communicate over RS485 interfac
peripherals/uart/nmea0183_parser	Obtaining GPS information by parsing NMEA0183 statemen

API Reference

Header File

- [components/driver/uart/include/driver/uart.h](#)

Functions

```
esp_err_t uart_driver_install(uart_port_t uart_num, int rx_buffer_size, int tx_buffer_size, int queue_size, QueueHandle_t *uart_queue, int intr_alloc_flags)
```

Install UART driver and set the UART to the default configuration.

UART ISR handler will be attached to the same CPU core that this function is running on.

Note

Rx_buffer_size should be greater than UART_FIFO_LEN. Tx_buffer_size should be either zero or greater than UART_FIFO_LEN.

Parameters:

- **uart_num** – UART port number, the max port number is (UART_NUM_MAX - 1).
- **rx_buffer_size** – UART RX ring buffer size.
- **tx_buffer_size** – UART TX ring buffer size. If set to zero, driver will not use TX buffer, TX function will block task until all data have been sent out.
- **queue_size** – UART event queue size/depth.

- **uart_queue** – UART event queue handle (out param). On success, a new queue handle is written here to provide access to UART events. If set to NULL, driver will not use an event queue.
- **intr_alloc_flags** – Flags used to allocate the interrupt. One or multiple (ORred) ESP_INTR_FLAG_* values. See esp_intr_alloc.h for more info. Do not set ESP_INTR_FLAG_IRAM here (the driver's ISR handler is not located in IRAM)

Returns:

- ESP_OK Success
- ESP_FAIL Parameter error

esp_err_t uart_driver_delete(uart_port_t uart_num)

Uninstall UART driver.

Parameters: **uart_num** – UART port number, the max port number is (UART_NUM_MAX -1).

Returns:

- ESP_OK Success
- ESP_FAIL Parameter error

bool uart_is_driver_installed(uart_port_t uart_num)

Checks whether the driver is installed or not.

Parameters: **uart_num** – UART port number, the max port number is (UART_NUM_MAX -1).

Returns:

- true driver is installed
- false driver is not installed

esp_err_t uart_set_word_length(uart_port_t uart_num, uart_word_length_t data_bit)

Set UART data bits.

Parameters:

- **uart_num** – UART port number, the max port number is (UART_NUM_MAX -1).
- **data_bit** – UART data bits

Returns:

- ESP_OK Success
- ESP_FAIL Parameter error


```
esp_err_t uart_get_word_length(uart_port_t uart_num, uart_word_length_t *data_bit)
```

Get the UART data bit configuration.

- Parameters:**
- **uart_num** – UART port number, the max port number is (UART_NUM_MAX -1).
 - **data_bit** – Pointer to accept value of UART data bits.

- Returns:**
- ESP_FAIL Parameter error
 - ESP_OK Success, result will be put in (*data_bit)

```
esp_err_t uart_set_stop_bits(uart_port_t uart_num, uart_stop_bits_t stop_bits)
```

Set UART stop bits.

- Parameters:**
- **uart_num** – UART port number, the max port number is (UART_NUM_MAX -1).
 - **stop_bits** – UART stop bits

- Returns:**
- ESP_OK Success
 - ESP_FAIL Fail

```
esp_err_t uart_get_stop_bits(uart_port_t uart_num, uart_stop_bits_t *stop_bits)
```

Get the UART stop bit configuration.

- Parameters:**
- **uart_num** – UART port number, the max port number is (UART_NUM_MAX -1).
 - **stop_bits** – Pointer to accept value of UART stop bits.

- Returns:**
- ESP_FAIL Parameter error
 - ESP_OK Success, result will be put in (*stop_bit)

```
esp_err_t uart_set_parity(uart_port_t uart_num, uart_parity_t parity_mode)
```

Set UART parity mode.

- Parameters:**
- **uart_num** – UART port number, the max port number is (UART_NUM_MAX -1).
 - **parity_mode** – the enum of uart parity configuration

- Returns:**
- ESP_FAIL Parameter error
 - ESP_OK Success

```
esp_err_t uart_get_parity(uart_port_t uart_num, uart_parity_t *parity_mode)
```

Get the UART parity mode configuration.

- Parameters:**
- **uart_num** – UART port number, the max port number is (UART_NUM_MAX -1).
 - **parity_mode** – Pointer to accept value of UART parity mode.

- Returns:**
- ESP_FAIL Parameter error
 - ESP_OK Success, result will be put in (*parity_mode)

```
esp_err_t uart_get_sclk_freq(uart_sclk_t sclk, uint32_t *out_freq_hz)
```

Get the frequency of a clock source for the UART.

- Parameters:**
- **sclk** – Clock source
 - **out_freq_hz** – [out] Output of frequency, in Hz

- Returns:**
- ESP_ERR_INVALID_ARG: if the clock source is not supported
 - otherwise ESP_OK

```
esp_err_t uart_set_baudrate(uart_port_t uart_num, uint32_t baudrate)
```

Set UART baud rate.

- Parameters:**
- **uart_num** – UART port number, the max port number is (UART_NUM_MAX -1).
 - **baudrate** – UART baud rate.

- Returns:**
- ESP_FAIL Parameter error
 - ESP_OK Success

```
esp_err_t uart_get_baudrate(uart_port_t uart_num, uint32_t *baudrate)
```

Get the UART baud rate configuration.

- Parameters:**
- **uart_num** – UART port number, the max port number is (UART_NUM_MAX -1).
 - **baudrate** – Pointer to accept value of UART baud rate

- Returns:**
- ESP_FAIL Parameter error
 - ESP_OK Success, result will be put in (*baudrate)

```
esp_err_t uart_set_line_inverse(uart_port_t uart_num, uint32_t inverse_mask)
```

Set UART line inverse mode.

- Parameters:**
- **uart_num** – UART port number, the max port number is (UART_NUM_MAX -1).
 - **inverse_mask** – Choose the wires that need to be inverted. Using the ORred mask of `uart_signal_inv_t`

- Returns:**
- ESP_OK Success
 - ESP_FAIL Parameter error

```
esp_err_t uart_set_hw_flow_ctrl(uart_port_t uart_num, uart_hw_flowcontrol_t flow_ctrl, uint8_t rx_thresh)
```

Set hardware flow control.

- Parameters:**
- **uart_num** – UART port number, the max port number is (UART_NUM_MAX -1).
 - **flow_ctrl** – Hardware flow control mode
 - **rx_thresh** – Threshold of Hardware RX flow control (0 ~ UART_FIFO_LEN). Only when UART_HW_FLOWCTRL_RTS is set, will the rx_thresh value be set.

- Returns:**
- ESP_OK Success
 - ESP_FAIL Parameter error

```
esp_err_t uart_set_sw_flow_ctrl(uart_port_t uart_num, bool enable, uint8_t rx_thresh_xon, uint8_t rx_thresh_xoff)
```

Set software flow control.

- Parameters:**
- **uart_num** – UART_NUM_0, UART_NUM_1 or UART_NUM_2
 - **enable** – switch on or off
 - **rx_thresh_xon** – low water mark
 - **rx_thresh_xoff** – high water mark

- Returns:**
- ESP_OK Success
 - ESP_FAIL Parameter error

```
esp_err_t uart_get_hw_flow_ctrl(uart_port_t uart_num, uart_hw_flowcontrol_t *flow_ctrl)
```

Get the UART hardware flow control configuration.

- Parameters:**
- **uart_num** – UART port number, the max port number is (UART_NUM_MAX -1).

- **flow_ctrl** – Option for different flow control mode.

Returns:

- ESP_FAIL Parameter error
- ESP_OK Success, result will be put in (*flow_ctrl)

```
esp_err_t uart_clear_intr_status(uart_port_t uart_num, uint32_t clr_mask)
```

Clear UART interrupt status.

- Parameters:**
- **uart_num** – UART port number, the max port number is (UART_NUM_MAX -1).
 - **clr_mask** – Bit mask of the interrupt status to be cleared.

Returns:

- ESP_OK Success
- ESP_FAIL Parameter error

```
esp_err_t uart_enable_intr_mask(uart_port_t uart_num, uint32_t enable_mask)
```

Set UART interrupt enable.

- Parameters:**
- **uart_num** – UART port number, the max port number is (UART_NUM_MAX -1).
 - **enable_mask** – Bit mask of the enable bits.

Returns:

- ESP_OK Success
- ESP_FAIL Parameter error

```
esp_err_t uart_disable_intr_mask(uart_port_t uart_num, uint32_t disable_mask)
```

Clear UART interrupt enable bits.

- Parameters:**
- **uart_num** – UART port number, the max port number is (UART_NUM_MAX -1).
 - **disable_mask** – Bit mask of the disable bits.

Returns:

- ESP_OK Success
- ESP_FAIL Parameter error

```
esp_err_t uart_enable_rx_intr(uart_port_t uart_num)
```

Enable UART RX interrupt (RX_FULL & RX_TIMEOUT INTERRUPT)

- Parameters:**
- **uart_num** – UART port number, the max port number is (UART_NUM_MAX -1).

Returns:

- ESP_OK Success
- ESP_FAIL Parameter error

esp_err_t uart_disable_rx_intr(uart_port_t uart_num)

Disable UART RX interrupt (RX_FULL & RX_TIMEOUT INTERRUPT)

Parameters: **uart_num** – UART port number, the max port number is (UART_NUM_MAX -1).

Returns:

- ESP_OK Success
- ESP_FAIL Parameter error

esp_err_t uart_disable_tx_intr(uart_port_t uart_num)

Disable UART TX interrupt (TX_FULL & TX_TIMEOUT INTERRUPT)

Parameters: **uart_num** – UART port number

Returns:

- ESP_OK Success
- ESP_FAIL Parameter error

esp_err_t uart_enable_tx_intr(uart_port_t uart_num, int enable, int thresh)

Enable UART TX interrupt (TX_FULL & TX_TIMEOUT INTERRUPT)

Parameters:

- **uart_num** – UART port number, the max port number is (UART_NUM_MAX -1).
- **enable** – 1: enable; 0: disable
- **thresh** – Threshold of TX interrupt, 0 ~ UART_FIFO_LEN

Returns:

- ESP_OK Success
- ESP_FAIL Parameter error

esp_err_t uart_set_pin(uart_port_t uart_num, int tx_io_num, int rx_io_num, int rts_io_num, int cts_io_num)

Assign signals of a UART peripheral to GPIO pins.

Note

If the GPIO number configured for a UART signal matches one of the IOMUX signals for that GPIO, the signal will be connected directly via the IOMUX. Otherwise the GPIO and

signal will be connected via the GPIO Matrix. For example, if on an ESP32 the call `uart_set_pin(0, 1, 3, -1, -1)` is performed, as GPIO1 is UART0's default TX pin and GPIO3 is UART0's default RX pin, both will be connected to respectively U0TXD and U0RXD through the IOMUX, totally bypassing the GPIO matrix. The check is performed on a per-pin basis. Thus, it is possible to have RX pin binded to a GPIO through the GPIO matrix, whereas TX is binded to its GPIO through the IOMUX.

Note

Internal signal can be output to multiple GPIO pads. Only one GPIO pad can connect with input signal.

- Parameters:**
- **uart_num** – UART port number, the max port number is (UART_NUM_MAX -1).
 - **tx_io_num** – UART TX pin GPIO number.
 - **rx_io_num** – UART RX pin GPIO number.
 - **rts_io_num** – UART RTS pin GPIO number.
 - **cts_io_num** – UART CTS pin GPIO number.

- Returns:**
- ESP_OK Success
 - ESP_FAIL Parameter error

esp_err_t uart_set_rts(uart_port_t uart_num, int level)

Manually set the UART RTS pin level.

Note

UART must be configured with hardware flow control disabled.

- Parameters:**
- **uart_num** – UART port number, the max port number is (UART_NUM_MAX -1).
 - **level** – 1: RTS output low (active); 0: RTS output high (block)

- Returns:**
- ESP_OK Success
 - ESP_FAIL Parameter error

esp_err_t uart_set_dtr(uart_port_t uart_num, int level)

Manually set the UART DTR pin level.

- Parameters:**
- **uart_num** – UART port number, the max port number is

(UART_NUM_MAX -1).

- **level** – 1: DTR output low; 0: DTR output high

Returns:

- ESP_OK Success
- ESP_FAIL Parameter error

```
esp_err_t uart_set_tx_idle_num(uart_port_t uart_num, uint16_t idle_num)
```

Set UART idle interval after tx FIFO is empty.

Parameters:

- **uart_num** – UART port number, the max port number is (UART_NUM_MAX -1).
- **idle_num** – idle interval after tx FIFO is empty(unit: the time it takes to send one bit under current baudrate)

Returns:

- ESP_OK Success
- ESP_FAIL Parameter error

```
esp_err_t uart_param_config(uart_port_t uart_num, const uart_config_t *uart_config)
```

Set UART configuration parameters.

Parameters:

- **uart_num** – UART port number, the max port number is (UART_NUM_MAX -1).
- **uart_config** – UART parameter settings

Returns:

- ESP_OK Success
- ESP_FAIL Parameter error

```
esp_err_t uart_intr_config(uart_port_t uart_num, const uart_intr_config_t *intr_conf)
```

Configure UART interrupts.

Parameters:

- **uart_num** – UART port number, the max port number is (UART_NUM_MAX -1).
- **intr_conf** – UART interrupt settings

Returns:

- ESP_OK Success
- ESP_FAIL Parameter error

```
esp_err_t uart_wait_tx_done(uart_port_t uart_num, TickType_t ticks_to_wait)
```

Wait until UART TX FIFO is empty.

- Parameters:**
- **uart_num** – UART port number, the max port number is (UART_NUM_MAX -1).
 - **ticks_to_wait** – Timeout, count in RTOS ticks

- Returns:**
- ESP_OK Success
 - ESP_FAIL Parameter error
 - ESP_ERR_TIMEOUT Timeout

```
int uart_tx_chars(uart_port_t uart_num, const char *buffer, uint32_t len)
```

Send data to the UART port from a given buffer and length.

This function will not wait for enough space in TX FIFO. It will just fill the available TX FIFO and return when the FIFO is full.

Note

This function should only be used when UART TX buffer is not enabled.

- Parameters:**
- **uart_num** – UART port number, the max port number is (UART_NUM_MAX -1).
 - **buffer** – data buffer address
 - **len** – data length to send

- Returns:**
- (-1) Parameter error
 - OTHERS (≥ 0) The number of bytes pushed to the TX FIFO

```
int uart_write_bytes(uart_port_t uart_num, const void *src, size_t size)
```

Send data to the UART port from a given buffer and length,.

If the UART driver's parameter 'tx_buffer_size' is set to zero: This function will not return until all the data have been sent out, or at least pushed into TX FIFO.

Otherwise, if the 'tx_buffer_size' > 0 , this function will return after copying all the data to tx ring buffer, UART ISR will then move data from the ring buffer to TX FIFO gradually.

- Parameters:**
- **uart_num** – UART port number, the max port number is (UART_NUM_MAX -1).
 - **src** – data buffer address
 - **size** – data length to send

- Returns:**
- (-1) Parameter error
 - OTHERS (≥ 0) The number of bytes pushed to the TX FIFO


```
int uart_write_bytes_with_break(uart_port_t uart_num, const void *src, size_t size, int brk_len)
```

Send data to the UART port from a given buffer and length,.

If the UART driver's parameter 'tx_buffer_size' is set to zero: This function will not return until all the data and the break signal have been sent out. After all data is sent out, send a break signal.

Otherwise, if the 'tx_buffer_size' > 0, this function will return after copying all the data to tx ring buffer, UART ISR will then move data from the ring buffer to TX FIFO gradually. After all data sent out, send a break signal.

- Parameters:**
- **uart_num** – UART port number, the max port number is (UART_NUM_MAX -1).
 - **src** – data buffer address
 - **size** – data length to send
 - **brk_len** – break signal duration(unit: the time it takes to send one bit at current baudrate)

- Returns:**
- (-1) Parameter error
 - OTHERS (>=0) The number of bytes pushed to the TX FIFO

```
int uart_read_bytes(uart_port_t uart_num, void *buf, uint32_t length, TickType_t ticks_to_wait)
```

UART read bytes from UART buffer.

- Parameters:**
- **uart_num** – UART port number, the max port number is (UART_NUM_MAX -1).
 - **buf** – pointer to the buffer.
 - **length** – data length
 - **ticks_to_wait** – sTimeout, count in RTOS ticks

- Returns:**
- (-1) Error
 - OTHERS (>=0) The number of bytes read from UART buffer

```
esp_err_t uart_flush(uart_port_t uart_num)
```

Alias of uart_flush_input. UART ring buffer flush. This will discard all data in the UART RX buffer.

Note

Instead of waiting the data sent out, this function will clear UART rx buffer. In order to send all the data in tx FIFO, we can use uart_wait_tx_done function.

Parameters: `uart_num` – UART port number, the max port number is (UART_NUM_MAX -1).

Returns:

- ESP_OK Success
- ESP_FAIL Parameter error

`esp_err_t uart_flush_input(uart_port_t uart_num)`

Clear input buffer, discard all the data is in the ring-buffer.

Note

In order to send all the data in tx FIFO, we can use `uart_wait_tx_done` function.

Parameters: `uart_num` – UART port number, the max port number is (UART_NUM_MAX -1).

Returns:

- ESP_OK Success
- ESP_FAIL Parameter error

`esp_err_t uart_get_buffered_data_len(uart_port_t uart_num, size_t *size)`

UART get RX ring buffer cached data length.

Parameters:

- `uart_num` – UART port number, the max port number is (UART_NUM_MAX -1).
- `size` – Pointer of `size_t` to accept cached data length

Returns:

- ESP_OK Success
- ESP_FAIL Parameter error

`esp_err_t uart_get_tx_buffer_free_size(uart_port_t uart_num, size_t *size)`

UART get TX ring buffer free space size.

Parameters:

- `uart_num` – UART port number, the max port number is (UART_NUM_MAX -1).
- `size` – Pointer of `size_t` to accept the free space size

Returns:

- ESP_OK Success
- ESP_ERR_INVALID_ARG Parameter error

esp_err_t uart_disable_pattern_det_intr(uart_port_t uart_num)

UART disable pattern detect function. Designed for applications like 'AT commands'. When the hardware detects a series of one same character, the interrupt will be triggered.

Parameters: **uart_num** – UART port number, the max port number is (UART_NUM_MAX -1).

Returns:

- ESP_OK Success
- ESP_FAIL Parameter error

esp_err_t uart_enable_pattern_det_baud_intr(uart_port_t uart_num, char pattern_chr, uint8_t chr_num, int chr_tout, int post_idle, int pre_idle)

UART enable pattern detect function. Designed for applications like 'AT commands'. When the hardware detect a series of one same character, the interrupt will be triggered.

Parameters:

- **uart_num** – UART port number.
- **pattern_chr** – character of the pattern.
- **chr_num** – number of the character, 8bit value.
- **chr_tout** – timeout of the interval between each pattern characters, 16bit value, unit is the baud-rate cycle you configured. When the duration is more than this value, it will not take this data as at_cmd char.
- **post_idle** – idle time after the last pattern character, 16bit value, unit is the baud-rate cycle you configured. When the duration is less than this value, it will not take the previous data as the last at_cmd char
- **pre_idle** – idle time before the first pattern character, 16bit value, unit is the baud-rate cycle you configured. When the duration is less than this value, it will not take this data as the first at_cmd char.

Returns:

- ESP_OK Success
- ESP_FAIL Parameter error

int uart_pattern_pop_pos(uart_port_t uart_num)

Return the nearest detected pattern position in buffer. The positions of the detected pattern are saved in a queue, this function will dequeue the first pattern position and move the pointer to next pattern position.

The following APIs will modify the pattern position info: uart_flush_input, uart_read_bytes, uart_driver_delete, uart_pop_pattern_pos It is the application's responsibility to ensure atomic access to the pattern queue and the rx data buffer when using pattern detect feature.

Note

If the RX buffer is full and flow control is not enabled, the detected pattern may not be found in the rx buffer due to overflow.

Parameters: **uart_num** – UART port number, the max port number is (UART_NUM_MAX -1).

Returns:

- (-1) No pattern found for current index or parameter error
- others the pattern position in rx buffer.

int **uart_pattern_get_pos**(**uart_port_t** **uart_num**)

Return the nearest detected pattern position in buffer. The positions of the detected pattern are saved in a queue, This function do nothing to the queue.

The following APIs will modify the pattern position info: **uart_flush_input**, **uart_read_bytes**, **uart_driver_delete**, **uart_pop_pattern_pos** It is the application's responsibility to ensure atomic access to the pattern queue and the rx data buffer when using pattern detect feature.

Note

If the RX buffer is full and flow control is not enabled, the detected pattern may not be found in the rx buffer due to overflow.

Parameters: **uart_num** – UART port number, the max port number is (UART_NUM_MAX -1).

Returns:

- (-1) No pattern found for current index or parameter error
- others the pattern position in rx buffer.

esp_err_t **uart_pattern_queue_reset**(**uart_port_t** **uart_num**, **int** **queue_length**)

Allocate a new memory with the given length to save record the detected pattern position in rx buffer.

Parameters:

- **uart_num** – UART port number, the max port number is (UART_NUM_MAX -1).
- **queue_length** – Max queue length for the detected pattern. If the queue length is not large enough, some pattern positions might be lost. Set this value to the maximum number of patterns that could be

saved in data buffer at the same time.

Returns:

- ESP_ERR_NO_MEM No enough memory
- ESP_ERR_INVALID_STATE Driver not installed
- ESP_FAIL Parameter error
- ESP_OK Success

```
esp_err_t uart_set_mode(uart_port_t uart_num, uart_mode_t mode)
```

UART set communication mode.

Note

This function must be executed after `uart_driver_install()`, when the driver object is initialized.

- Parameters:**
- **uart_num** – Uart number to configure, the max port number is (UART_NUM_MAX -1).
 - **mode** – UART mode to set

Returns:

- ESP_OK Success
- ESP_ERR_INVALID_ARG Parameter error

```
esp_err_t uart_set_rx_full_threshold(uart_port_t uart_num, int threshold)
```

Set uart threshold value for RX fifo full.

Note

If application is using higher baudrate and it is observed that bytes in hardware RX fifo are overwritten then this threshold can be reduced

- Parameters:**
- **uart_num** – UART_NUM_0, UART_NUM_1 or UART_NUM_2
 - **threshold** – Threshold value above which RX fifo full interrupt is generated

Returns:

- ESP_OK Success
- ESP_ERR_INVALID_ARG Parameter error
- ESP_ERR_INVALID_STATE Driver is not installed

```
esp_err_t uart_set_tx_empty_threshold(uart_port_t uart_num, int threshold)
```

Set uart threshold values for TX fifo empty.

- Parameters:**
- **uart_num** – UART_NUM_0, UART_NUM_1 or UART_NUM_2
 - **threshold** – Threshold value below which TX fifo empty interrupt is generated

- Returns:**
- ESP_OK Success
 - ESP_ERR_INVALID_ARG Parameter error
 - ESP_ERR_INVALID_STATE Driver is not installed

```
esp_err_t uart_set_rx_timeout(uart_port_t uart_num, const uint8_t tout_thresh)
```

UART set threshold timeout for TOUT feature.

- Parameters:**
- **uart_num** – Uart number to configure, the max port number is (UART_NUM_MAX -1).
 - **tout_thresh** – This parameter defines timeout threshold in uart symbol periods. The maximum value of threshold is 126. tout_thresh = 1, defines TOUT interrupt timeout equal to transmission time of one symbol (~11 bit) on current baudrate. If the time is expired the UART_RXFIFO_TOUT_INT interrupt is triggered. If tout_thresh == 0, the TOUT feature is disabled.

- Returns:**
- ESP_OK Success
 - ESP_ERR_INVALID_ARG Parameter error
 - ESP_ERR_INVALID_STATE Driver is not installed

```
esp_err_t uart_get_collision_flag(uart_port_t uart_num, bool *collision_flag)
```

Returns collision detection flag for RS485 mode Function returns the collision detection flag into variable pointed by collision_flag. *collision_flag = true, if collision detected else it is equal to false. This function should be executed when actual transmission is completed (after uart_write_bytes()).

- Parameters:**
- **uart_num** – Uart number to configure the max port number is (UART_NUM_MAX -1).
 - **collision_flag** – Pointer to variable of type bool to return collision flag.

- Returns:**
- ESP_OK Success
 - ESP_ERR_INVALID_ARG Parameter error

```
esp_err_t uart_set_wakeup_threshold(uart_port_t uart_num, int wakeup_threshold)
```

Set the number of RX pin signal edges for light sleep wakeup.

UART can be used to wake up the system from light sleep. This feature works by counting the number of positive edges on RX pin and comparing the count to the threshold. When the count exceeds the threshold, system is woken up from light sleep. This function allows setting the threshold value.

Stop bit and parity bits (if enabled) also contribute to the number of edges. For example, letter 'a' with ASCII code 97 is encoded as 0100001101 on the wire (with 8n1 configuration), start and stop bits included. This sequence has 3 positive edges (transitions from 0 to 1). Therefore, to wake up the system when 'a' is sent, set wakeup_threshold=3.

The character that triggers wakeup is not received by UART (i.e. it can not be obtained from UART FIFO). Depending on the baud rate, a few characters after that will also not be received. Note that when the chip enters and exits light sleep mode, APB frequency will be changing. To ensure that UART has correct Baud rate all the time, it is necessary to select a source clock which has a fixed frequency and remains active during sleep. For the supported clock sources of the chips, please refer to `uart_sclk_t` or `soc_periph_uart_clk_src_legacy_t`

Note

in ESP32, the wakeup signal can only be input via IO_MUX (i.e. GPIO3 should be configured as function_1 to wake up UART0, GPIO9 should be configured as function_5 to wake up UART1), UART2 does not support light sleep wakeup feature.

Parameters:

- **uart_num** – UART number, the max port number is (UART_NUM_MAX -1).
- **wakeup_threshold** – number of RX edges for light sleep wakeup, value is 3 .. 0x3ff.

Returns:

- ESP_OK on success
- ESP_ERR_INVALID_ARG if uart_num is incorrect or wakeup_threshold is outside of [3, 0x3ff] range.

```
esp_err_t uart_get_wakeup_threshold(uart_port_t uart_num, int *out_wakeup_threshold)
```

Get the number of RX pin signal edges for light sleep wakeup.

See description of `uart_set_wakeup_threshold` for the explanation of UART wakeup feature.

Parameters:

- **uart_num** – UART number, the max port number is (UART_NUM_MAX -1).
- **out_wakeup_threshold** – [out] output, set to the current value of wakeup threshold for the given UART.

Returns:

- ESP_OK on success
- ESP_ERR_INVALID_ARG if out_wakeup_threshold is NULL

esp_err_t uart_wait_tx_idle_polling(uart_port_t uart_num)

Wait until UART tx memory empty and the last char send ok (polling mode).

•

Returns:

- ESP_OK on success
- ESP_ERR_INVALID_ARG Parameter error
- ESP_FAIL Driver not installed

Parameters: **uart_num** – UART number

esp_err_t uart_set_loop_back(uart_port_t uart_num, bool loop_back_en)

Configure TX signal loop back to RX module, just for the test usage.

•

Returns:

- ESP_OK on success
- ESP_ERR_INVALID_ARG Parameter error
- ESP_FAIL Driver not installed

Parameters:

- **uart_num** – UART number
- **loop_back_en** – Set ture to enable the loop back function, else set it false.

void uart_set_always_rx_timeout(uart_port_t uart_num, bool always_rx_timeout_en)

Configure behavior of UART RX timeout interrupt.

When always_rx_timeout is true, timeout interrupt is triggered even if FIFO is full. This function can cause extra timeout interrupts triggered only to send the timeout event. Call this function only if you want to ensure timeout interrupt will always happen after a byte stream.

Parameters:

- **uart_num** – UART number
- **always_rx_timeout_en** – Set to false enable the default behavior of timeout interrupt, set it to true to always trigger timeout interrupt.

Structures

struct uart_intr_config_t

UART interrupt configuration parameters for uart_intr_config function.

Public Members

uint32_t intr_enable_mask

UART interrupt enable mask, choose from UART_XXXX_INT_ENA_M under UART_INT_ENA_REG(i), connect with bit-or operator

uint8_t rx_timeout_thresh

UART timeout interrupt threshold (unit: time of sending one byte)

uint8_t txfifo_empty_intr_thresh

UART TX empty interrupt threshold.

uint8_t rxfifo_full_thresh

UART RX full interrupt threshold.

struct uart_event_t

Event structure used in UART event queue.

Public Members

uart_event_type_t type

UART event type

size_t size

UART data size for UART_DATA event

bool timeout_flag

UART data read timeout flag for UART_DATA event (no new data received during configured RX TOUT) If the event is caused by FIFO-full interrupt, then there will be no event with the timeout flag before the next byte coming.

Macros

UART_NUM_0

UART port 0

UART_NUM_1

UART port 1

UART_NUM_2

UART port 2

UART_NUM_MAX

UART port max

UART_PIN_NO_CHANGE

UART_FIFO_LEN

Length of the UART HW FIFO.

UART_BITRATE_MAX

Maximum configurable bitrate.

Type Definitions

typedef **intr_handle_t** **uart_isr_handle_t**

Enumerations

enum **uart_event_type_t**

UART event types used in the ring buffer.

Values:

enumerator **UART_DATA**

UART data event

enumerator **UART_BREAK**

UART break event

enumerator **UART_BUFFER_FULL**

UART RX buffer full event

enumerator **UART_FIFO_OVF**

UART FIFO overflow event

enumerator **UART_FRAME_ERR**

UART RX frame error event

enumerator **UART_PARITY_ERR**

UART RX parity event

enumerator **UART_DATA_BREAK**

UART TX data and break event

enumerator **UART_PATTERN_DET**

UART pattern detected

enumerator **UART_EVENT_MAX**

UART event max index

Header File

- [components/hal/include/hal/uart_types.h](#)

Structures

struct **uart_at_cmd_t**

UART AT cmd char configuration parameters Note that this function may different on different chip. Please refer to the TRM at configration.

Public Members

uint8_t cmd_char

UART AT cmd char

uint8_t char_num

AT cmd char repeat number

uint32_t gap_tout

gap time(in baud-rate) between AT cmd char

uint32_t pre_idle

the idle time(in baud-rate) between the non AT char and first AT char

uint32_t post_idle

the idle time(in baud-rate) between the last AT char and the none AT char

struct uart_sw_flowctrl_t

UART software flow control configuration parameters.

Public Members

uint8_t xon_char

Xon flow control char

uint8_t xoff_char

Xoff flow control char

uint8_t xon_thrd

If the software flow control is enabled and the data amount in rxfifo is less than xon_thrd, an xon_char will be sent

uint8_t xoff_thrd

If the software flow control is enabled and the data amount in rxfifo is more than xoff_thrd, an xoff_char will be sent

struct uart_config_t

UART configuration parameters for uart_param_config function.

Public Members

int baud_rate

UART baud rate

uart_word_length_t data_bits

UART byte size

uart_parity_t parity

UART parity mode

uart_stop_bits_t stop_bits

UART stop bits

uart_hw_flowcontrol_t flow_ctrl

UART HW flow control mode (cts/rts)

uint8_t rx_flow_ctrl_thresh

UART HW RTS threshold

uart_sclk_t source_clk

UART source clock selection

Type Definitions

typedef int uart_port_t

UART port number, can be UART_NUM_0 ~ (UART_NUM_MAX -1).

typedef soc_periph_uart_clk_src_legacy_t uart_sclk_t

UART source clock.

Enumerations

enum uart_mode_t

UART mode selection.

Values:

enumerator UART_MODE_UART

mode: regular UART mode

enumerator UART_MODE_RS485_HALF_DUPLEX

mode: half duplex RS485 UART mode control by RTS pin

enumerator UART_MODE_IRDA

mode: IRDA UART mode

enumerator UART_MODE_RS485_COLLISION_DETECT

mode: RS485 collision detection UART mode (used for test purposes)

enumerator UART_MODE_RS485_APP_CTRL

mode: application control RS485 UART mode (used for test purposes)

enum uart_word_length_t

UART word length constants.

Values:

enumerator UART_DATA_5_BITS

word length: 5bits

enumerator UART_DATA_6_BITS

word length: 6bits

enumerator UART_DATA_7_BITS

word length: 7bits

enumerator UART_DATA_8_BITS

word length: 8bits

enumerator UART_DATA_BITS_MAX

enum uart_stop_bits_t

UART stop bits number.

Values:

enumerator UART_STOP_BITS_1

stop bit: 1bit

enumerator UART_STOP_BITS_1_5

stop bit: 1.5bits

enumerator UART_STOP_BITS_2

stop bit: 2bits

enumerator **UART_STOP_BITS_MAX**

enum **uart_parity_t**

UART parity constants.

Values:

enumerator **UART_PARITY_DISABLE**

Disable UART parity

enumerator **UART_PARITY_EVEN**

Enable UART even parity

enumerator **UART_PARITY_ODD**

Enable UART odd parity

enum **uart_hw_flowcontrol_t**

UART hardware flow control modes.

Values:

enumerator **UART_HW_FLOWCTRL_DISABLE**

disable hardware flow control

enumerator **UART_HW_FLOWCTRL_RTS**

enable RX hardware flow control (rts)

enumerator **UART_HW_FLOWCTRL_CTS**

enable TX hardware flow control (cts)

enumerator **UART_HW_FLOWCTRL_CTS_RTS**

enable hardware flow control

enumerator **UART_HW_FLOWCTRL_MAX**

enum **uart_signal_inv_t**

UART signal bit map.

Values:

enumerator **UART_SIGNAL_INV_DISABLE**

Disable UART signal inverse

enumerator **UART_SIGNAL_IRDA_TX_INV**

inverse the UART irda_tx signal

enumerator **UART_SIGNAL_IRDA_RX_INV**

inverse the UART irda_rx signal

enumerator **UART_SIGNAL_RXD_INV**

inverse the UART rxd signal

enumerator **UART_SIGNAL_CTS_INV**

inverse the UART cts signal

enumerator **UART_SIGNAL_DSR_INV**

inverse the UART dsr signal

enumerator **UART_SIGNAL_TXD_INV**

inverse the UART txd signal

enumerator **UART_SIGNAL_RTS_INV**

inverse the UART rts signal

enumerator **UART_SIGNAL_DTR_INV**

inverse the UART dtr signal

GPIO Lookup Macros

The UART peripherals have dedicated IO_MUX pins to which they are connected directly. However, signals can also be routed to other pins using the less direct GPIO matrix. To use direct routes, you need to know which pin is a dedicated IO_MUX pin for a UART channel. GPIO Lookup Macros simplify the process of finding and assigning IO_MUX pins. You choose a macro based on either the IO_MUX pin number, or a required UART channel name, and the macro will return the matching counterpart for you. See some examples below.

Note

These macros are useful if you need very high UART baud rates (over 40 MHz), which means you will have to use IO_MUX pins only. In other cases, these macros can be ignored, and you can use the GPIO Matrix as it allows you to configure any GPIO pin for any UART function.

1. `UART_NUM_2_TXD_DIRECT_GPIO_NUM` returns the IO_MUX pin number of UART channel 2 TXD pin (pin 17)
2. `UART_GPIO19_DIRECT_CHANNEL` returns the UART number of GPIO 19 when connected to the UART peripheral via IO_MUX (this is UART_NUM_0)
3. `UART_CTS_GPIO19_DIRECT_CHANNEL` returns the UART number of GPIO 19 when used as the UART CTS pin via IO_MUX (this is UART_NUM_0). It is similar to the above macro but specifies the pin function which is also part of the IO_MUX assignment.

Header File

- [components/soc/esp32/include/soc/uart_channel.h](#)

Macros

`UART_GPIO1_DIRECT_CHANNEL`

`UART_NUM_0_TXD_DIRECT_GPIO_NUM`

`UART_GPIO3_DIRECT_CHANNEL`

`UART_NUM_0_RXD_DIRECT_GPIO_NUM`

`UART_GPIO19_DIRECT_CHANNEL`

`UART_NUM_0_CTS_DIRECT_GPIO_NUM`

`UART_GPIO22_DIRECT_CHANNEL`

`UART_NUM_0_RTS_DIRECT_GPIO_NUM`

`UART_TXD_GPIO1_DIRECT_CHANNEL`

UART_RXD_GPIO3_DIRECT_CHANNEL

UART_CTS_GPIO19_DIRECT_CHANNEL

UART_RTS_GPIO22_DIRECT_CHANNEL

UART_GPIO10_DIRECT_CHANNEL

UART_NUM_1_TXD_DIRECT_GPIO_NUM

UART_GPIO9_DIRECT_CHANNEL

UART_NUM_1_RXD_DIRECT_GPIO_NUM

UART_GPIO6_DIRECT_CHANNEL

UART_NUM_1_CTS_DIRECT_GPIO_NUM

UART_GPIO11_DIRECT_CHANNEL

UART_NUM_1_RTS_DIRECT_GPIO_NUM

UART_TXD_GPIO10_DIRECT_CHANNEL

UART_RXD_GPIO9_DIRECT_CHANNEL

UART_CTS_GPIO6_DIRECT_CHANNEL

UART_RTS_GPIO11_DIRECT_CHANNEL

UART_GPIO17_DIRECT_CHANNEL

UART_NUM_2_TXD_DIRECT_GPIO_NUM

UART_GPIO16_DIRECT_CHANNEL

UART_NUM_2_RXD_DIRECT_GPIO_NUM

UART_GPIO8_DIRECT_CHANNEL

UART_NUM_2_CTS_DIRECT_GPIO_NUM

UART_GPIO7_DIRECT_CHANNEL

UART_NUM_2_RTS_DIRECT_GPIO_NUM

UART_TXD_GPIO17_DIRECT_CHANNEL

UART_RXD_GPIO16_DIRECT_CHANNEL

UART_CTS_GPIO8_DIRECT_CHANNEL

UART_RTS_GPIO7_DIRECT_CHANNEL

[Provide feedback about this document](#)

[← Previous](#)

[Next →](#)

© Copyright 2016 - 2023, Espressif Systems (Shanghai) Co., Ltd.

Built with [Sphinx](#) using a [theme](#) based on [Read the Docs Sphinx Theme](#).

[Download PDF](#)

**Note**

There is a newer bugfix release of this version. The latest bugfix release is [v5.1.6](#)

GPIO & RTC GPIO

[\[中文\]](#)

GPIO Summary

The ESP32 chip features 34 physical GPIO pins (GPIO0 ~ GPIO19, GPIO21 ~ GPIO23, GPIO25 ~ GPIO27, and GPIO32 ~ GPIO39). Each pin can be used as a general-purpose I/O, or be connected to an internal peripheral signal. Through IO MUX, RTC IO MUX and the GPIO matrix, peripheral input signals can be from any IO pins, and peripheral output signals can be routed to any IO pins. Together these modules provide highly configurable I/O. For more details, see *ESP32 Technical Reference Manual > IO MUX and GPIO Matrix (GPIO, IO_MUX)* [[PDF](#)].

The table below provides more information on pin usage, and please note the comments in the table for GPIOs with restrictions.

GPIO	Analog Function	RTC GPIO	Comments
GPIO0	ADC2_CH1	RTC_GPIO11	Strapping pin
GPIO1			TXD
GPIO2	ADC2_CH2	RTC_GPIO12	Strapping pin
GPIO3			RXD
GPIO4	ADC2_CH0	RTC_GPIO10	
GPIO5			Strapping pin
GPIO6			SPI0/1
GPIO7			SPI0/1
GPIO8			SPI0/1

GPIO	Analog Function	RTC GPIO	Comments
GPIO9			SPI0/1
GPIO10			SPI0/1
GPIO11			SPI0/1
GPIO12	ADC2_CH5	RTC_GPIO15	Strapping pin; JTAG
GPIO13	ADC2_CH4	RTC_GPIO14	JTAG
GPIO14	ADC2_CH6	RTC_GPIO16	JTAG
GPIO15	ADC2_CH3	RTC_GPIO13	Strapping pin; JTAG
GPIO16			SPI0/1
GPIO17			SPI0/1
GPIO18			
GPIO19			
GPIO21			
GPIO22			
GPIO23			
GPIO25	ADC2_CH8	RTC_GPIO6	
GPIO26	ADC2_CH9	RTC_GPIO7	
GPIO27	ADC2_CH7	RTC_GPIO17	
GPIO32	ADC1_CH4	RTC_GPIO9	
GPIO33	ADC1_CH5	RTC_GPIO8	
GPIO34	ADC1_CH6	RTC_GPIO4	GPI
GPIO35	ADC1_CH7	RTC_GPIO5	GPI
GPIO36	ADC1_CH0	RTC_GPIO0	GPI
GPIO37	ADC1_CH1	RTC_GPIO1	GPI
GPIO38	ADC1_CH2	RTC_GPIO2	GPI
GPIO39	ADC1_CH3	RTC_GPIO3	GPI

Note

- Strapping pin: GPIO0, GPIO2, GPIO5, GPIO12 (MTDI), and GPIO15 (MTDO) are strapping pins. For more information, please refer to [ESP32 datasheet](#).
- SPI0/1: GPIO6-11 and GPIO16-17 are usually connected to the SPI flash and PSRAM integrated on the module and therefore should not be used for other purposes.

- JTAG: GPIO12-15 are usually used for inline debug.
- GPI: GPIO34-39 can only be set as input mode and do not have software-enabled pullup or pulldown functions.
- TXD & RXD are usually used for flashing and debugging.
- ADC2: ADC2 pins cannot be used when Wi-Fi is used. So, if you are having trouble getting the value from an ADC2 GPIO while using Wi-Fi, you may consider using an ADC1 GPIO instead, which should solve your problem. For more details, please refer to [Hardware Limitations of ADC Continuous Mode](#) and [Hardware Limitations of ADC Oneshot Mode](#).
- Please do not use the interrupt of GPIO36 and GPIO39 when using ADC or Wi-Fi and Bluetooth with sleep mode enabled. Please refer to [ESP32 ECO and Workarounds for Bugs](#) > Section 3.11 for the detailed description of the issue.

There is also separate “RTC GPIO” support, which functions when GPIOs are routed to the “RTC” low-power and analog subsystem. These pin functions can be used when:

- In Deep-sleep mode
- The [Ultra Low Power co-processor](#) is running
- Analog functions such as ADC/DAC/etc are in use.

Application Example

- GPIO output and input interrupt example: [peripherals/gpio/generic_gpio](#).

API Reference - Normal GPIO

Header File

- [components/driver/gpio/include/driver/gpio.h](#)

Functions

```
esp_err_t gpio_config(const gpio_config_t *pGPIOConfig)
```

GPIO common configuration.

Configure GPIO's Mode, pull-up, PullDown, IntrType

Parameters: pGPIOConfig – Pointer to GPIO configure struct

Returns:

- ESP_OK success

- ESP_ERR_INVALID_ARG Parameter error

`esp_err_t gpio_reset_pin(gpio_num_t gpio_num)`

Reset an gpio to default state (select gpio function, enable pullup and disable input and output).

Note

This function also configures the IOMUX for this pin to the GPIO function, and disconnects any other peripheral output configured via GPIO Matrix.

Parameters: `gpio_num` – GPIO number.

Returns: Always return ESP_OK.

`esp_err_t gpio_set_intr_type(gpio_num_t gpio_num, gpio_int_type_t intr_type)`

GPIO set interrupt trigger type.

Parameters:

- `gpio_num` – GPIO number. If you want to set the trigger type of e.g. of GPIO16, `gpio_num` should be `GPIO_NUM_16` (16);
- `intr_type` – Interrupt type, select from `gpio_int_type_t`

Returns:

- ESP_OK Success
- ESP_ERR_INVALID_ARG Parameter error

`esp_err_t gpio_intr_enable(gpio_num_t gpio_num)`

Enable GPIO module interrupt signal.

Note

ESP32: Please do not use the interrupt of GPIO36 and GPIO39 when using ADC or Wi-Fi and Bluetooth with sleep mode enabled. Please refer to the comments of `adc1_get_raw`. Please refer to Section 3.11 of [ESP32 ECO and Workarounds for Bugs](#) for the description of this issue.

Parameters: `gpio_num` – GPIO number. If you want to enable an interrupt on e.g. GPIO16, `gpio_num` should be `GPIO_NUM_16` (16);

Returns:

- ESP_OK Success
- ESP_ERR_INVALID_ARG Parameter error

esp_err_t gpio_intr_disable(gpio_num_t gpio_num)

Disable GPIO module interrupt signal.

Note

This function is allowed to be executed when Cache is disabled within ISR context, by enabling `CONFIG_GPIO_CTRL_FUNC_IN_IRAM`

Parameters: **gpio_num** – GPIO number. If you want to disable the interrupt of e.g. GPIO16, gpio_num should be GPIO_NUM_16 (16);

Returns:

- ESP_OK success
- ESP_ERR_INVALID_ARG Parameter error

esp_err_t gpio_set_level(gpio_num_t gpio_num, uint32_t level)

GPIO set output level.

Note

This function is allowed to be executed when Cache is disabled within ISR context, by enabling `CONFIG_GPIO_CTRL_FUNC_IN_IRAM`

Parameters:

- **gpio_num** – GPIO number. If you want to set the output level of e.g. GPIO16, gpio_num should be GPIO_NUM_16 (16);
- **level** – Output level. 0: low ; 1: high

Returns:

- ESP_OK Success
- ESP_ERR_INVALID_ARG GPIO number error

int gpio_get_level(gpio_num_t gpio_num)

GPIO get input level.

Warning

If the pad is not configured for input (or input and output) the returned value is always 0.

Parameters: **gpio_num** – GPIO number. If you want to get the logic level of e.g. pin GPIO16, gpio_num should be GPIO_NUM_16 (16);

Returns:

- 0 the GPIO input level is 0
- 1 the GPIO input level is 1

```
esp_err_t gpio_set_direction(gpio_num_t gpio_num, gpio_mode_t mode)
```

GPIO set direction.

Configure GPIO direction,such as output_only,input_only,output_and_input

- Parameters:**
- **gpio_num** – Configure GPIO pins number, it should be GPIO number. If you want to set direction of e.g. GPIO16, gpio_num should be GPIO_NUM_16 (16);
 - **mode** – GPIO direction

Returns:

- ESP_OK Success
- ESP_ERR_INVALID_ARG GPIO error

```
esp_err_t gpio_set_pull_mode(gpio_num_t gpio_num, gpio_pull_mode_t pull)
```

Configure GPIO pull-up/pull-down resistors.

Note

ESP32: Only pins that support both input & output have integrated pull-up and pull-down resistors. Input-only GPIOs 34-39 do not.

- Parameters:**
- **gpio_num** – GPIO number. If you want to set pull up or down mode for e.g. GPIO16, gpio_num should be GPIO_NUM_16 (16);
 - **pull** – GPIO pull up/down mode.

Returns:

- ESP_OK Success
- ESP_ERR_INVALID_ARG : Parameter error

```
esp_err_t gpio_wakeup_enable(gpio_num_t gpio_num, gpio_int_type_t intr_type)
```

Enable GPIO wake-up function.

- Parameters:**
- **gpio_num** – GPIO number.
 - **intr_type** – GPIO wake-up type. Only GPIO_INTR_LOW_LEVEL or GPIO_INTR_HIGH_LEVEL can be used.

Returns:

- ESP_OK Success
- ESP_ERR_INVALID_ARG Parameter error

`esp_err_t gpio_wakeup_disable(gpio_num_t gpio_num)`

Disable GPIO wake-up function.

Parameters: `gpio_num` – GPIO number

Returns:

- ESP_OK Success
- ESP_ERR_INVALID_ARG Parameter error

`esp_err_t gpio_isr_register(void (*fn)(void*), void *arg, int intr_alloc_flags, gpio_isr_handle_t *handle)`

Register GPIO interrupt handler, the handler is an ISR. The handler will be attached to the same CPU core that this function is running on.

This ISR function is called whenever any GPIO interrupt occurs. See the alternative `gpio_install_isr_service()` and `gpio_isr_handler_add()` API in order to have the driver support per-GPIO ISRs.

To disable or remove the ISR, pass the returned handle to the [interrupt allocation functions](#).

Parameters:

- **fn** – Interrupt handler function.
- **arg** – Parameter for handler function
- **intr_alloc_flags** – Flags used to allocate the interrupt. One or multiple (ORred) ESP_INTR_FLAG_* values. See `esp_intr_alloc.h` for more info.
- **handle** – Pointer to return handle. If non-NULL, a handle for the interrupt will be returned here.

Returns:

- ESP_OK Success ;
- ESP_ERR_INVALID_ARG GPIO error
- ESP_ERR_NOT_FOUND No free interrupt found with the specified flags

`esp_err_t gpio_pullup_en(gpio_num_t gpio_num)`

Enable pull-up on GPIO.

Parameters: `gpio_num` – GPIO number

Returns:

- ESP_OK Success
- ESP_ERR_INVALID_ARG Parameter error

`esp_err_t gpio_pullup_dis(gpio_num_t gpio_num)`

Disable pull-up on GPIO.

Parameters: `gpio_num` – GPIO number

Returns:

- ESP_OK Success
- ESP_ERR_INVALID_ARG Parameter error

```
esp_err_t gpio_pulldown_en(gpio_num_t gpio_num)
```

Enable pull-down on GPIO.

Parameters: `gpio_num` – GPIO number

Returns:

- ESP_OK Success
- ESP_ERR_INVALID_ARG Parameter error

```
esp_err_t gpio_pulldown_dis(gpio_num_t gpio_num)
```

Disable pull-down on GPIO.

Parameters: `gpio_num` – GPIO number

Returns:

- ESP_OK Success
- ESP_ERR_INVALID_ARG Parameter error

```
esp_err_t gpio_install_isr_service(int intr_alloc_flags)
```

Install the GPIO driver's ETS_GPIO_INTR_SOURCE ISR handler service, which allows per-pin GPIO interrupt handlers.

This function is incompatible with `gpio_isr_register()` - if that function is used, a single global ISR is registered for all GPIO interrupts. If this function is used, the ISR service provides a global GPIO ISR and individual pin handlers are registered via the `gpio_isr_handler_add()` function.

Parameters: `intr_alloc_flags` – Flags used to allocate the interrupt. One or multiple (ORred) ESP_INTR_FLAG_* values. See `esp_intr_alloc.h` for more info.

Returns:

- ESP_OK Success
- ESP_ERR_NO_MEM No memory to install this service
- ESP_ERR_INVALID_STATE ISR service already installed.
- ESP_ERR_NOT_FOUND No free interrupt found with the specified flags
- ESP_ERR_INVALID_ARG GPIO error

```
void gpio_uninstall_isr_service(void)
```

Uninstall the driver's GPIO ISR service, freeing related resources.

```
esp_err_t gpio_isr_handler_add(gpio_num_t gpio_num, gpio_isr_t isr_handler, void *args)
```

Add ISR handler for the corresponding GPIO pin.

Call this function after using `gpio_install_isr_service()` to install the driver's GPIO ISR handler service.

The pin ISR handlers no longer need to be declared with `IRAM_ATTR`, unless you pass the `ESP_INTR_FLAG_IRAM` flag when allocating the ISR in `gpio_install_isr_service()`.

This ISR handler will be called from an ISR. So there is a stack size limit (configurable as "ISR stack size" in menuconfig). This limit is smaller compared to a global GPIO interrupt handler due to the additional level of indirection.

- Parameters:**
- **gpio_num** – GPIO number
 - **isr_handler** – ISR handler function for the corresponding GPIO number.
 - **args** – parameter for ISR handler.

- Returns:**
- `ESP_OK` Success
 - `ESP_ERR_INVALID_STATE` Wrong state, the ISR service has not been initialized.
 - `ESP_ERR_INVALID_ARG` Parameter error

```
esp_err_t gpio_isr_handler_remove(gpio_num_t gpio_num)
```

Remove ISR handler for the corresponding GPIO pin.

- Parameters:** **gpio_num** – GPIO number

- Returns:**
- `ESP_OK` Success
 - `ESP_ERR_INVALID_STATE` Wrong state, the ISR service has not been initialized.
 - `ESP_ERR_INVALID_ARG` Parameter error

```
esp_err_t gpio_set_drive_capability(gpio_num_t gpio_num, gpio_drive_cap_t strength)
```

Set GPIO pad drive capability.

- Parameters:**
- **gpio_num** – GPIO number, only support output GPIOs
 - **strength** – Drive capability of the pad

Returns:

- ESP_OK Success
- ESP_ERR_INVALID_ARG Parameter error

```
esp_err_t gpio_get_drive_capability(gpio_num_t gpio_num, gpio_drive_cap_t *strength)
```

Get GPIO pad drive capability.

Parameters:

- **gpio_num** – GPIO number, only support output GPIOs
- **strength** – Pointer to accept drive capability of the pad

Returns:

- ESP_OK Success
- ESP_ERR_INVALID_ARG Parameter error

```
esp_err_t gpio_hold_en(gpio_num_t gpio_num)
```

Enable gpio pad hold function.

When a GPIO is set to hold, its state is latched at that moment and will not change when the internal signal or the IO MUX/GPIO configuration is modified (including input enable, output enable, output value, function, and drive strength values). This function can be used to retain the state of GPIOs when the chip or system is reset, for example, when watchdog time-out or Deep-sleep events are triggered.

This function works in both input and output modes, and only applicable to output-capable GPIOs. If this function is enabled: in output mode: the output level of the GPIO will be locked and can not be changed. in input mode: the input read value can still reflect the changes of the input signal.

However, on ESP32/S2/C3/S3/C2, this function cannot be used to hold the state of a digital GPIO during Deep-sleep. Even if this function is enabled, the digital GPIO will be reset to its default state when the chip wakes up from Deep-sleep. If you want to hold the state of a digital GPIO during Deep-sleep, please call `gpio_deep_sleep_hold_en`.

Power down or call `gpio_hold_dis` will disable this function.

Parameters:

gpio_num – GPIO number, only support output-capable GPIOs

Returns:

- ESP_OK Success
- ESP_ERR_NOT_SUPPORTED Not support pad hold function

```
esp_err_t gpio_hold_dis(gpio_num_t gpio_num)
```

Disable gpio pad hold function.

When the chip is woken up from Deep-sleep, the gpio will be set to the default mode, so, the gpio will output the default level if this function is called. If you don't want the level changes, the gpio should be configured to a known state before this function is called. e.g. If you hold gpio18 high during Deep-sleep, after the chip is woken up and `gpio_hold_dis` is called, gpio18 will output low level(because gpio18 is input mode by default). If you don't want this behavior, you should configure gpio18 as output mode and set it to high level before calling `gpio_hold_dis`.

Parameters: `gpio_num` – GPIO number, only support output-capable GPIOs

Returns:

- ESP_OK Success
- ESP_ERR_NOT_SUPPORTED Not support pad hold function

`void gpio_deep_sleep_hold_en(void)`

Enable all digital gpio pads hold function during Deep-sleep.

Enabling this feature makes all digital gpio pads be at the holding state during Deep-sleep. The state of each pad holds is its active configuration (not pad's sleep configuration!).

Note that this pad hold feature only works when the chip is in Deep-sleep mode. When the chip is in active mode, the digital gpio state can be changed freely even you have called this function.

After this API is being called, the digital gpio Deep-sleep hold feature will work during every sleep process. You should call `gpio_deep_sleep_hold_dis` to disable this feature.

`void gpio_deep_sleep_hold_dis(void)`

Disable all digital gpio pads hold function during Deep-sleep.

`void gpio_iomux_in(uint32_t gpio_num, uint32_t signal_idx)`

SOC_GPIO_SUPPORT_HOLD_SINGLE_IO_IN_DSLP.

Set pad input to a peripheral signal through the IOMUX.

Parameters:

- `gpio_num` – GPIO number of the pad.
- `signal_idx` – Peripheral signal id to input. One of the `*_IN_IDX` signals in `soc/gpio_sig_map.h`.

`void gpio_iomux_out(uint8_t gpio_num, int func, bool oen_inv)`

Set peripheral output to an GPIO pad through the IOMUX.

Parameters:

- `gpio_num` – gpio_num GPIO number of the pad.

- **func** – The function number of the peripheral pin to output pin. One of the `FUNC_X_*` of specified pin (X) in `soc/io_mux_reg.h`.
- **oen_inv** – True if the output enable needs to be inverted, otherwise False.

`esp_err_t gpio_sleep_sel_en(gpio_num_t gpio_num)`

Enable SLP_SEL to change GPIO status automatically in lightsleep.

Parameters: `gpio_num` – GPIO number of the pad.

Returns:

- ESP_OK Success

`esp_err_t gpio_sleep_sel_dis(gpio_num_t gpio_num)`

Disable SLP_SEL to change GPIO status automatically in lightsleep.

Parameters: `gpio_num` – GPIO number of the pad.

Returns:

- ESP_OK Success

`esp_err_t gpio_sleep_set_direction(gpio_num_t gpio_num, gpio_mode_t mode)`

GPIO set direction at sleep.

Configure GPIO direction,such as output_only,input_only,output_and_input

Parameters:

- **gpio_num** – Configure GPIO pins number, it should be GPIO number. If you want to set direction of e.g. GPIO16, gpio_num should be GPIO_NUM_16 (16);
- **mode** – GPIO direction

Returns:

- ESP_OK Success
- ESP_ERR_INVALID_ARG GPIO error

`esp_err_t gpio_sleep_set_pull_mode(gpio_num_t gpio_num, gpio_pull_mode_t pull)`

Configure GPIO pull-up/pull-down resistors at sleep.

Note

ESP32: Only pins that support both input & output have integrated pull-up and pull-down resistors. Input-only GPIOs 34-39 do not.

- Parameters:**
- **gpio_num** – GPIO number. If you want to set pull up or down mode for e.g. GPIO16, gpio_num should be GPIO_NUM_16 (16);
 - **pull** – GPIO pull up/down mode.
- Returns:**
- ESP_OK Success
 - ESP_ERR_INVALID_ARG : Parameter error

Structures

`struct gpio_config_t`

Configuration parameters of GPIO pad for gpio_config function.

Public Members

`uint64_t pin_bit_mask`

GPIO pin: set with bit mask, each bit maps to a GPIO

`gpio_mode_t mode`

GPIO mode: set input/output mode

`gpio_pullup_t pull_up_en`

GPIO pull-up

`gpio_pulldown_t pull_down_en`

GPIO pull-down

`gpio_int_type_t intr_type`

GPIO interrupt type

Macros

`GPIO_PIN_COUNT`

`GPIO_IS_VALID_GPIO(gpio_num)`

Check whether it is a valid GPIO number.

`GPIO_IS_VALID_OUTPUT_GPIO(gpio_num)`

Check whether it can be a valid GPIO number of output mode.


```
GPIO_IS_VALID_DIGITAL_IO_PAD(gpio\_num)
```

Check whether it can be a valid digital I/O pad.

Type Definitions

```
typedef intr\_handle\_t gpio_isr_handle_t
```

```
typedef void (*gpio\_isr\_t)(void *arg)
```

GPIO interrupt handler.

Param arg: User registered data

Header File

- [components/hal/include/hal/gpio_types.h](#)

Macros

```
GPIO_PIN_REG_0
```

```
GPIO_PIN_REG_1
```

```
GPIO_PIN_REG_2
```

```
GPIO_PIN_REG_3
```

```
GPIO_PIN_REG_4
```

```
GPIO_PIN_REG_5
```

```
GPIO_PIN_REG_6
```

```
GPIO_PIN_REG_7
```

```
GPIO_PIN_REG_8
```

GPIO_PIN_REG_9

GPIO_PIN_REG_10

GPIO_PIN_REG_11

GPIO_PIN_REG_12

GPIO_PIN_REG_13

GPIO_PIN_REG_14

GPIO_PIN_REG_15

GPIO_PIN_REG_16

GPIO_PIN_REG_17

GPIO_PIN_REG_18

GPIO_PIN_REG_19

GPIO_PIN_REG_20

GPIO_PIN_REG_21

GPIO_PIN_REG_22

GPIO_PIN_REG_23

GPIO_PIN_REG_24

GPIO_PIN_REG_25

GPIO_PIN_REG_26

GPIO_PIN_REG_27

GPIO_PIN_REG_28

GPIO_PIN_REG_29

GPIO_PIN_REG_30

GPIO_PIN_REG_31

GPIO_PIN_REG_32

GPIO_PIN_REG_33

GPIO_PIN_REG_34

GPIO_PIN_REG_35

GPIO_PIN_REG_36

GPIO_PIN_REG_37

GPIO_PIN_REG_38

GPIO_PIN_REG_39

GPIO_PIN_REG_40

GPIO_PIN_REG_41

GPIO_PIN_REG_42

GPIO_PIN_REG_43

GPIO_PIN_REG_44

GPIO_PIN_REG_45

GPIO_PIN_REG_46

GPIO_PIN_REG_47

GPIO_PIN_REG_48

Enumerations

***enum* gpio_port_t**

Values:

enumerator **GPIO_PORT_0**

enumerator **GPIO_PORT_MAX**

***enum* gpio_num_t**

Values:

enumerator **GPIO_NUM_NC**

Use to signal not connected to S/W

enumerator **GPIO_NUM_0**

GPIO0, input and output

enumerator **GPIO_NUM_1**

GPIO1, input and output

enumerator **GPIO_NUM_2**

GPIO2, input and output

enumerator **GPIO_NUM_3**

GPIO3, input and output

enumerator **GPIO_NUM_4**

GPIO4, input and output

enumerator **GPIO_NUM_5**

GPIO5, input and output

enumerator **GPIO_NUM_6**

GPIO6, input and output

enumerator **GPIO_NUM_7**

GPIO7, input and output

enumerator **GPIO_NUM_8**

GPIO8, input and output

enumerator **GPIO_NUM_9**

GPIO9, input and output

enumerator **GPIO_NUM_10**

GPIO10, input and output

enumerator **GPIO_NUM_11**

GPIO11, input and output

enumerator **GPIO_NUM_12**

GPIO12, input and output

enumerator **GPIO_NUM_13**

GPIO13, input and output

enumerator **GPIO_NUM_14**

GPIO14, input and output

enumerator **GPIO_NUM_15**

GPIO15, input and output

enumerator **GPIO_NUM_16**

GPIO16, input and output

enumerator **GPIO_NUM_17**

GPIO17, input and output

enumerator **GPIO_NUM_18**

GPIO18, input and output

enumerator **GPIO_NUM_19**

GPIO19, input and output

enumerator **GPIO_NUM_20**

GPIO20, input and output

enumerator **GPIO_NUM_21**

GPIO21, input and output

enumerator **GPIO_NUM_22**

GPIO22, input and output

enumerator **GPIO_NUM_23**

GPIO23, input and output

enumerator **GPIO_NUM_25**

GPIO25, input and output

enumerator **GPIO_NUM_26**

GPIO26, input and output

enumerator **GPIO_NUM_27**

GPIO27, input and output

enumerator **GPIO_NUM_28**

GPIO28, input and output

enumerator **GPIO_NUM_29**

GPIO29, input and output

enumerator **GPIO_NUM_30**

GPIO30, input and output

enumerator **GPIO_NUM_31**

GPIO31, input and output

enumerator **GPIO_NUM_32**

GPIO32, input and output

enumerator **GPIO_NUM_33**

GPIO33, input and output

enumerator **GPIO_NUM_34**

GPIO34, input mode only

enumerator **GPIO_NUM_35**

GPIO35, input mode only

enumerator **GPIO_NUM_36**

GPIO36, input mode only

enumerator **GPIO_NUM_37**

GPIO37, input mode only

enumerator **GPIO_NUM_38**

GPIO38, input mode only

enumerator **GPIO_NUM_39**

GPIO39, input mode only

enumerator **GPIO_NUM_MAX**

enum **gpio_int_type_t**

Values:

enumerator **GPIO_INTR_DISABLE**

Disable GPIO interrupt

enumerator **GPIO_INTR_POSEDGE**

GPIO interrupt type : rising edge

enumerator **GPIO_INTR_NEGEDGE**

GPIO interrupt type : falling edge

enumerator **GPIO_INTR_ANYEDGE**

GPIO interrupt type : both rising and falling edge

enumerator **GPIO_INTR_LOW_LEVEL**

GPIO interrupt type : input low level trigger

enumerator **GPIO_INTR_HIGH_LEVEL**

GPIO interrupt type : input high level trigger

enumerator **GPIO_INTR_MAX**

enum **gpio_mode_t**

Values:

enumerator **GPIO_MODE_DISABLE**

GPIO mode : disable input and output

enumerator **GPIO_MODE_INPUT**

GPIO mode : input only

enumerator **GPIO_MODE_OUTPUT**

GPIO mode : output only mode

enumerator **GPIO_MODE_OUTPUT_OD**

GPIO mode : output only with open-drain mode

enumerator **GPIO_MODE_INPUT_OUTPUT_OD**

GPIO mode : output and input with open-drain mode

enumerator **GPIO_MODE_INPUT_OUTPUT**

GPIO mode : output and input mode

enum **gpio_pullup_t**

Values:

enumerator **GPIO_PULLUP_DISABLE**

Disable GPIO pull-up resistor

enumerator **GPIO_PULLUP_ENABLE**

Enable GPIO pull-up resistor

enum **gpiopulldown_t**

Values:

enumerator **GPIO_PULLDOWN_DISABLE**

Disable GPIO pull-down resistor

enumerator **GPIO_PULLDOWN_ENABLE**

Enable GPIO pull-down resistor

enum **gpio_pull_mode_t**

Values:

enumerator **GPIO_PULLUP_ONLY**

Pad pull up

enumerator **GPIO_PULLDOWN_ONLY**

Pad pull down

enumerator **GPIO_PULLUP_PULLDOWN**

Pad pull up + pull down

enumerator **GPIO_FLOATING**

Pad floating

enum gpio_drive_cap_t

Values:

enumerator GPIO_DRIVE_CAP_0

Pad drive capability: weak

enumerator GPIO_DRIVE_CAP_1

Pad drive capability: stronger

enumerator GPIO_DRIVE_CAP_2

Pad drive capability: medium

enumerator GPIO_DRIVE_CAP_DEFAULT

Pad drive capability: medium

enumerator GPIO_DRIVE_CAP_3

Pad drive capability: strongest

enumerator GPIO_DRIVE_CAP_MAX

enum gpio_hys_ctrl_mode_t

Available option for configuring hysteresis feature of GPIOs.

Values:

enumerator GPIO_HYS_CTRL_EFUSE

Pad input hysteresis ctrl by efuse

enumerator GPIO_HYS_SOFT_ENABLE

Pad input hysteresis enable by software

enumerator GPIO_HYS_SOFT_DISABLE

Pad input hysteresis disable by software

API Reference - RTC GPIO

Header File

- [components/driver/gpio/include/driver/rtc_io.h](#)

Functions

bool rtc_gpio_is_valid_gpio(gpio_num_t gpio_num)

Determine if the specified GPIO is a valid RTC GPIO.

Parameters: **gpio_num** – GPIO number

Returns: true if GPIO is valid for RTC GPIO use. false otherwise.

int rtc_io_number_get(gpio_num_t gpio_num)

Get RTC IO index number by gpio number.

Parameters: **gpio_num** – GPIO number

Returns: >=0: Index of rtcio. -1 : The gpio is not rtcio.

esp_err_t rtc_gpio_init(gpio_num_t gpio_num)

Init a GPIO as RTC GPIO.

This function must be called when initializing a pad for an analog function.

Parameters: **gpio_num** – GPIO number (e.g. GPIO_NUM_12)

Returns:

- ESP_OK success
- ESP_ERR_INVALID_ARG GPIO is not an RTC IO

esp_err_t rtc_gpio_deinit(gpio_num_t gpio_num)

Init a GPIO as digital GPIO.

Parameters: **gpio_num** – GPIO number (e.g. GPIO_NUM_12)

Returns:

- ESP_OK success
- ESP_ERR_INVALID_ARG GPIO is not an RTC IO

uint32_t rtc_gpio_get_level(gpio_num_t gpio_num)

Get the RTC IO input level.

Parameters: **gpio_num** – GPIO number (e.g. GPIO_NUM_12)

Returns:

- 1 High level
- 0 Low level
- ESP_ERR_INVALID_ARG GPIO is not an RTC IO

esp_err_t rtc_gpio_set_level(gpio_num_t gpio_num, uint32_t level)

Set the RTC IO output level.

- Parameters:**
- **gpio_num** – GPIO number (e.g. GPIO_NUM_12)
 - **level** – output level

Returns:

- ESP_OK Success
- ESP_ERR_INVALID_ARG GPIO is not an RTC IO

esp_err_t rtc_gpio_set_direction(gpio_num_t gpio_num, rtc_gpio_mode_t mode)

RTC GPIO set direction.

Configure RTC GPIO direction, such as output only, input only, output and input.

- Parameters:**
- **gpio_num** – GPIO number (e.g. GPIO_NUM_12)
 - **mode** – GPIO direction

Returns:

- ESP_OK Success
- ESP_ERR_INVALID_ARG GPIO is not an RTC IO

esp_err_t rtc_gpio_set_direction_in_sleep(gpio_num_t gpio_num, rtc_gpio_mode_t mode)

RTC GPIO set direction in deep sleep mode or disable sleep status (default). In some application scenarios, IO needs to have another states during deep sleep.

NOTE: ESP32 support INPUT_ONLY mode. ESP32S2 support INPUT_ONLY, OUTPUT_ONLY, INPUT_OUTPUT mode.

- Parameters:**
- **gpio_num** – GPIO number (e.g. GPIO_NUM_12)
 - **mode** – GPIO direction

Returns:

- ESP_OK Success
- ESP_ERR_INVALID_ARG GPIO is not an RTC IO

esp_err_t rtc_gpio_pullup_en(gpio_num_t gpio_num)

RTC GPIO pullup enable.

This function only works for RTC IOs. In general, call `gpio_pullup_en`, which will work both for normal GPIOs and RTC IOs.

Parameters: `gpio_num` – GPIO number (e.g. `GPIO_NUM_12`)

Returns:

- `ESP_OK` Success
- `ESP_ERR_INVALID_ARG` GPIO is not an RTC IO

```
esp_err_t rtc_gpio_pulldown_en(gpio_num_t gpio_num)
```

RTC GPIO pulldown enable.

This function only works for RTC IOs. In general, call `gpio_pulldown_en`, which will work both for normal GPIOs and RTC IOs.

Parameters: `gpio_num` – GPIO number (e.g. `GPIO_NUM_12`)

Returns:

- `ESP_OK` Success
- `ESP_ERR_INVALID_ARG` GPIO is not an RTC IO

```
esp_err_t rtc_gpio_pullup_dis(gpio_num_t gpio_num)
```

RTC GPIO pullup disable.

This function only works for RTC IOs. In general, call `gpio_pullup_dis`, which will work both for normal GPIOs and RTC IOs.

Parameters: `gpio_num` – GPIO number (e.g. `GPIO_NUM_12`)

Returns:

- `ESP_OK` Success
- `ESP_ERR_INVALID_ARG` GPIO is not an RTC IO

```
esp_err_t rtc_gpio_pulldown_dis(gpio_num_t gpio_num)
```

RTC GPIO pulldown disable.

This function only works for RTC IOs. In general, call `gpio_pulldown_dis`, which will work both for normal GPIOs and RTC IOs.

Parameters: `gpio_num` – GPIO number (e.g. `GPIO_NUM_12`)

Returns:

- `ESP_OK` Success
- `ESP_ERR_INVALID_ARG` GPIO is not an RTC IO

```
esp_err_t rtc_gpio_set_drive_capability(gpio_num_t gpio_num, gpio_drive_cap_t strength)
```

Set RTC GPIO pad drive capability.

- Parameters:**
- **gpio_num** – GPIO number, only support output GPIOs
 - **strength** – Drive capability of the pad

- Returns:**
- ESP_OK Success
 - ESP_ERR_INVALID_ARG Parameter error

```
esp_err_t rtc_gpio_get_drive_capability(gpio_num_t gpio_num, gpio_drive_cap_t *strength)
```

Get RTC GPIO pad drive capability.

- Parameters:**
- **gpio_num** – GPIO number, only support output GPIOs
 - **strength** – Pointer to accept drive capability of the pad

- Returns:**
- ESP_OK Success
 - ESP_ERR_INVALID_ARG Parameter error

```
esp_err_t rtc_gpio_hold_en(gpio_num_t gpio_num)
```

Enable hold function on an RTC IO pad.

Enabling HOLD function will cause the pad to latch current values of input enable, output enable, output value, function, drive strength values. This function is useful when going into light or deep sleep mode to prevent the pin configuration from changing.

- Parameters:** **gpio_num** – GPIO number (e.g. GPIO_NUM_12)

- Returns:**
- ESP_OK Success
 - ESP_ERR_INVALID_ARG GPIO is not an RTC IO

```
esp_err_t rtc_gpio_hold_dis(gpio_num_t gpio_num)
```

Disable hold function on an RTC IO pad.

Disabling hold function will allow the pad receive the values of input enable, output enable, output value, function, drive strength from RTC_IO peripheral.

- Parameters:** **gpio_num** – GPIO number (e.g. GPIO_NUM_12)

- Returns:**
- ESP_OK Success
 - ESP_ERR_INVALID_ARG GPIO is not an RTC IO

```
esp_err_t rtc_gpio_force_hold_en_all(void)
```

Enable force hold signal for all RTC IOs.

Each RTC pad has a “force hold” input signal from the RTC controller. If this signal is set, pad latches current values of input enable, function, output enable, and other signals which come from the RTC mux. Force hold signal is enabled before going into deep sleep for pins which are used for EXT1 wakeup.

```
esp_err_t rtc_gpio_force_hold_dis_all(void)
```

Disable force hold signal for all RTC IOs.

```
esp_err_t rtc_gpio_wakeup_enable(gpio_num_t gpio_num, gpio_int_type_t intr_type)
```

Enable wakeup from sleep mode using specific GPIO.

- Parameters:**
- **gpio_num** – GPIO number
 - **intr_type** – Wakeup on high level (GPIO_INTR_HIGH_LEVEL) or low level (GPIO_INTR_LOW_LEVEL)
- Returns:**
- ESP_OK on success
 - ESP_ERR_INVALID_ARG if gpio_num is not an RTC IO, or intr_type is not one of GPIO_INTR_HIGH_LEVEL, GPIO_INTR_LOW_LEVEL.

```
esp_err_t rtc_gpio_wakeup_disable(gpio_num_t gpio_num)
```

Disable wakeup from sleep mode using specific GPIO.

- Parameters:** **gpio_num** – GPIO number
- Returns:**
- ESP_OK on success
 - ESP_ERR_INVALID_ARG if gpio_num is not an RTC IO

Macros

```
RTC_GPIO_IS_VALID_GPIO(gpio_num)
```

Header File

- [components/hal/include/hal/rtc_io_types.h](#)

Enumerations

enum rtc_gpio_mode_t

RTCIO output/input mode type.

Values:

enumerator RTC_GPIO_MODE_INPUT_ONLY

Pad input

enumerator RTC_GPIO_MODE_OUTPUT_ONLY

Pad output

enumerator RTC_GPIO_MODE_INPUT_OUTPUT

Pad input + output

enumerator RTC_GPIO_MODE_DISABLED

Pad (output + input) disable

enumerator RTC_GPIO_MODE_OUTPUT_OD

Pad open-drain output

enumerator RTC_GPIO_MODE_INPUT_OUTPUT_OD

Pad input + open-drain output

[Provide feedback about this document](#)

[← Previous](#)

[Next →](#)

© Copyright 2016 - 2023, Espressif Systems (Shanghai) Co., Ltd.

Built with [Sphinx](#) using a [theme](#) based on [Read the Docs Sphinx Theme](#).

[Download PDF](#)

[🏠](#) » [API Reference](#) » [Peripherals API](#) »

Analog to Digital Converter (ADC) Oneshot Mode Driver

[🔗 Edit on GitHub](#)**Note**

There is a newer bugfix release of this version. The latest bugfix release is [v5.1.6](#)

Analog to Digital Converter (ADC) Oneshot Mode Driver

[\[中文\]](#)

Introduction

The Analog to Digital Converter is integrated on the chip and is capable of measuring analog signals from specific analog IO pins.

ESP32 has two ADC unit(s), which can be used in scenario(s) like:

- Generate one-shot ADC conversion result
- Generate continuous ADC conversion results

This guide introduces ADC oneshot mode conversion.

Functional Overview

The following sections of this document cover the typical steps to install and operate an ADC:

- [Resource Allocation](#) - covers which parameters should be set up to get an ADC handle and how to recycle the resources when ADC finishes working.
- [Unit Configuration](#) - covers the parameters that should be set up to configure the ADC unit, so as to get ADC conversion raw result.
- [Read Conversion Result](#) - covers how to get ADC conversion raw result.
- [Hardware Limitations](#) - describes the ADC-related hardware limitations.

- [Power Management](#) - covers power management-related information.
- [IRAM Safe](#) - describes tips on how to read ADC conversion raw results when the cache is disabled.
- [Thread Safety](#) - lists which APIs are guaranteed to be thread-safe by the driver.
- [Kconfig Options](#) - lists the supported Kconfig options that can be used to make a different effect on driver behavior.

Resource Allocation

The ADC oneshot mode driver is implemented based on ESP32 SAR ADC module. Different ESP chips might have different numbers of independent ADCs. From the oneshot mode driver's point of view, an ADC instance is represented by `adc_oneshot_unit_handle_t`.

To install an ADC instance, set up the required initial configuration structure

`adc_oneshot_unit_init_cfg_t`:

- `adc_oneshot_unit_init_cfg_t::unit_id` selects the ADC. Please refer to the [datasheet](#) to know dedicated analog IOs for this ADC.
- `adc_oneshot_unit_init_cfg_t::clk_src` selects the source clock of the ADC. If set to 0, the driver will fall back to using a default clock source, see `adc_oneshot_clk_src_t` to know the details.
- `adc_oneshot_unit_init_cfg_t::ulp_mode` sets if the ADC will be working under ULP mode.

After setting up the initial configurations for the ADC, call `adc_oneshot_new_unit()` with the prepared `adc_oneshot_unit_init_cfg_t`. This function will return an ADC unit handle if the allocation is successful.

This function may fail due to various errors such as invalid arguments, insufficient memory, etc. Specifically, when the to-be-allocated ADC instance is registered already, this function will return `ESP_ERR_NOT_FOUND` error. Number of available ADC(s) is recorded by `SOC_ADC_PERIPH_NUM`.

If a previously created ADC instance is no longer required, you should recycle the ADC instance by calling `adc_oneshot_del_unit()`, related hardware and software resources will be recycled as well.

Create an ADC Unit Handle Under Normal Oneshot Mode

```
adc_oneshot_unit_handle_t adc1_handle;
adc_oneshot_unit_init_cfg_t init_config1 = {
    .unit_id = ADC_UNIT_1,
    .ulp_mode = ADC_ULP_MODE_DISABLE,
};
ESP_ERROR_CHECK(adc_oneshot_new_unit(&init_config1, &adc1_handle));
```

Recycle the ADC Unit

```
ESP_ERROR_CHECK(adc_oneshot_del_unit(adc1_handle));
```

Unit Configuration

After an ADC instance is created, set up the `adc_oneshot_chan_cfg_t` to configure ADC IOs to measure analog signal:

- `adc_oneshot_chan_cfg_t::atten`, ADC attenuation. Refer to [TRM](#) > [On-Chip Sensor and Analog Signal Processing](#).
- `adc_oneshot_chan_cfg_t::bitwidth`, the bitwidth of the raw conversion result.

Note

For the IO corresponding ADC channel number, check [datasheet](#) to know the ADC IOs.

Additionally, `adc_continuous_io_to_channel()` and `adc_continuous_channel_to_io()` can be used to know the ADC channels and ADC IOs.

To make these settings take effect, call `adc_oneshot_config_channel()` with the above configuration structure. You should specify an ADC channel to be configured as well. Function `adc_oneshot_config_channel()` can be called multiple times to configure different ADC channels. The Driver will save each of these channel configurations internally.

Configure Two ADC Channels

```
adc_oneshot_chan_cfg_t config = {
    .bitwidth = ADC_BITWIDTH_DEFAULT,
    .atten = ADC_ATTEN_DB_11,
};
ESP_ERROR_CHECK(adc_oneshot_config_channel(adc1_handle, EXAMPLE_ADC1_CHAN0, &config));
ESP_ERROR_CHECK(adc_oneshot_config_channel(adc1_handle, EXAMPLE_ADC1_CHAN1, &config));
```

Read Conversion Result

After above configurations, the ADC is ready to measure the analog signal(s) from the configured ADC channel(s). Call `adc_oneshot_read()` to get the conversion raw result of an ADC channel.

- `adc_oneshot_read()` is safe to use. ADC(s) are shared by some other drivers/peripherals, see [Hardware Limitations](#). This function uses mutexes to avoid concurrent hardware usage. Therefore, this function should not be used in an ISR context. This function may fail when the ADC is in use by other drivers/peripherals, and return `ESP_ERR_TIMEOUT`. Under this condition, the ADC raw result is invalid.

This function will fail due to invalid arguments.

The ADC conversion results read from this function are raw data. To calculate the voltage based on the ADC raw results, this formula can be used:

$$V_{out} = D_{out} * V_{max} / D_{max} \tag{1}$$

where:

Vout	Digital output result, standing for the voltage.
Dout	ADC raw digital reading result.
Vmax	Maximum measurable input analog voltage, this is related to the ADC attenuation, please refer to the hardware manual.
Dmax	Maximum of the output ADC raw digital reading result, which is 2^bitwidth, where bitwidth is the resolution of the ADC.

To do further calibration to convert the ADC raw result to voltage in mV, please refer to calibration doc [Analog to Digital Converter \(ADC\) Calibration Driver](#).

Read Raw Result

```
ESP_ERROR_CHECK(adc_oneshot_read(adc1_handle, EXAMPLE_ADC1_CHAN0, &adc_raw[0][0]));
ESP_LOGI(TAG, "ADC%d Channel[%d] Raw Data: %d", ADC_UNIT_1 + 1, EXAMPLE_ADC1_CHAN0, adc_raw[0][0]);

ESP_ERROR_CHECK(adc_oneshot_read(adc1_handle, EXAMPLE_ADC1_CHAN1, &adc_raw[0][1]));
ESP_LOGI(TAG, "ADC%d Channel[%d] Raw Data: %d", ADC_UNIT_1 + 1, EXAMPLE_ADC1_CHAN1, adc_raw[0][1]);
```

Hardware Limitations

- Random Number Generator (RNG) uses ADC as an input source. When ADC `adc_oneshot_read()` works, the random number generated from RNG will be less random.
- A specific ADC unit can only work under one operating mode at any one time, either continuous mode or oneshot mode. `adc_oneshot_read()` has provided the protection.

- ADC2 is also used by Wi-Fi. `adc_oneshot_read()` has provided protection between the Wi-Fi driver and ADC oneshot mode driver.
- ESP32-DevKitC: GPIO0 cannot be used in oneshot mode, because the DevKit has used it for auto-flash.
- ESP-WROVER-KIT: GPIO 0, 2, 4, and 15 cannot be used due to external connections for different purposes.

Power Management

When power management is enabled, i.e., `CONFIG_PM_ENABLE` is on, the system clock frequency may be adjusted when the system is in an idle state. However, the ADC oneshot mode driver works in a polling routine, the `adc_oneshot_read()` will poll the CPU until the function returns. During this period of time, the task in which ADC oneshot mode driver resides will not be blocked. Therefore the clock frequency is stable when reading.

IRAM Safe

By default, all the ADC oneshot mode driver APIs are not supposed to be run when the Cache is disabled. Cache may be disabled due to many reasons, such as Flash writing/erasing, OTA, etc. If these APIs execute when the Cache is disabled, you will probably see errors like `Illegal Instruction` or `Load/Store Prohibited`.

Thread Safety

- `adc_oneshot_new_unit()`
- `adc_oneshot_config_channel()`
- `adc_oneshot_read()`

Above functions are guaranteed to be thread-safe. Therefore, you can call them from different RTOS tasks without protection by extra locks.

- `adc_oneshot_del_unit()` is not thread-safe. Besides, concurrently calling this function may result in failures of the above thread-safe APIs.

Kconfig Options

- `CONFIG_ADC_ONESHOT_CTRL_FUNC_IN_IRAM` controls where to place the ADC fast read function (IRAM or Flash), see [IRAM Safe](#) for more details.

Application Examples

- ADC oneshot mode example: [peripherals/adc/oneshot_read](#).

API Reference

Header File

- [components/hal/include/hal/adc_types.h](#)

Structures

`struct adc_digi_pattern_config_t`

ADC digital controller pattern configuration.

Public Members

`uint8_t atten`

Attenuation of this ADC channel.

`uint8_t channel`

ADC channel.

`uint8_t unit`

ADC unit.

`uint8_t bit_width`

ADC output bit width.

`struct adc_digi_output_data_t`

ADC digital controller (DMA mode) output data format. Used to analyze the acquired ADC (DMA) data.

Note

ESP32: Only `type1` is valid. ADC2 does not support DMA mode.

Note

ESP32-S2: Member `channel` can be used to judge the validity of the ADC data, because the role of the arbiter may get invalid ADC data.

Public Members

`uint16_t data`

ADC real output data info. Resolution: 12 bit.

ADC real output data info. Resolution: 11 bit.

`uint16_t channel`

ADC channel index info.

ADC channel index info. For ESP32-S2: If (channel < ADC_CHANNEL_MAX), The data is valid. If (channel > ADC_CHANNEL_MAX), The data is invalid.

`struct adc_digi_output_data_t::[anonymous] type1`

ADC type1

`uint16_t unit`

ADC unit index info. 0: ADC1; 1: ADC2.

`struct adc_digi_output_data_t::[anonymous] type2`

When the configured output format is 11bit.

`uint16_t val`

Raw data value

Type Definitions

```
typedef soc_periph_adc_digi_clk_src_t adc_one_shot_clk_src_t
```

Clock source type of oneshot mode which uses digital controller.

```
typedef soc_periph_adc_digi_clk_src_t adc_continuous_clk_src_t
```

Clock source type of continuous mode which uses digital controller.

Enumerations

```
enum adc_unit_t
```

ADC unit.

Values:

enumerator **ADC_UNIT_1**

SAR ADC 1.

enumerator **ADC_UNIT_2**

SAR ADC 2.

enum **adc_channel_t**

ADC channels.

Values:

enumerator **ADC_CHANNEL_0**

ADC channel.

enumerator **ADC_CHANNEL_1**

ADC channel.

enumerator **ADC_CHANNEL_2**

ADC channel.

enumerator **ADC_CHANNEL_3**

ADC channel.

enumerator **ADC_CHANNEL_4**

ADC channel.

enumerator **ADC_CHANNEL_5**

ADC channel.

enumerator **ADC_CHANNEL_6**

ADC channel.

enumerator **ADC_CHANNEL_7**

ADC channel.

enumerator **ADC_CHANNEL_8**

ADC channel.

enumerator **ADC_CHANNEL_9**

ADC channel.

enum **adc_atten_t**

ADC attenuation parameter. Different parameters determine the range of the ADC.

Values:

enumerator **ADC_ATTEN_DB_0**

No input attenuation, ADC can measure up to approx.

enumerator **ADC_ATTEN_DB_2_5**

The input voltage of ADC will be attenuated extending the range of measurement by about 2.5 dB (1.33 x)

enumerator **ADC_ATTEN_DB_6**

The input voltage of ADC will be attenuated extending the range of measurement by about 6 dB (2 x)

enumerator **ADC_ATTEN_DB_11**

The input voltage of ADC will be attenuated extending the range of measurement by about 11 dB (3.55 x)

enum **adc_bitwidth_t**

Values:

enumerator **ADC_BITWIDTH_DEFAULT**

Default ADC output bits, max supported width will be selected.

enumerator **ADC_BITWIDTH_9**

ADC output width is 9Bit.

enumerator **ADC_BITWIDTH_10**

ADC output width is 10Bit.

enumerator **ADC_BITWIDTH_11**

ADC output width is 11Bit.

enumerator **ADC_BITWIDTH_12**

ADC output width is 12Bit.

enumerator **ADC_BITWIDTH_13**

ADC output width is 13Bit.

enum **adc_ulp_mode_t**

Values:

enumerator **ADC_ULP_MODE_DISABLE**

ADC ULP mode is disabled.

enumerator **ADC_ULP_MODE_FSM**

ADC is controlled by ULP FSM.

enumerator **ADC_ULP_MODE_RISCV**

ADC is controlled by ULP RISCV.

enum **adc_digi_convert_mode_t**

ADC digital controller (DMA mode) work mode.

Values:

enumerator **ADC_CONV_SINGLE_UNIT_1**

Only use ADC1 for conversion.

enumerator **ADC_CONV_SINGLE_UNIT_2**

Only use ADC2 for conversion.

enumerator **ADC_CONV_BOTH_UNIT**

Use Both ADC1 and ADC2 for conversion simultaneously.

enumerator **ADC_CONV_ALTER_UNIT**

Use both ADC1 and ADC2 for conversion by turn. e.g. ADC1 -> ADC2 -> ADC1 -> ADC2

enum adc_digi_output_format_t

ADC digital controller (DMA mode) output data format option.

Values:

enumerator ADC_DIGI_OUTPUT_FORMAT_TYPE1

See `adc_digi_output_data_t.type1`

enumerator ADC_DIGI_OUTPUT_FORMAT_TYPE2

See `adc_digi_output_data_t.type2`

enum adc_digi_iir_filter_t

ADC IIR Filter ID.

Values:

enumerator ADC_DIGI_IIR_FILTER_0

Filter 0.

enumerator ADC_DIGI_IIR_FILTER_1

Filter 1.

enum adc_digi_iir_filter_coeff_t

IIR Filter Coefficient.

Values:

enumerator ADC_DIGI_IIR_FILTER_COEFF_2

The filter coefficient is 2.

enumerator ADC_DIGI_IIR_FILTER_COEFF_4

The filter coefficient is 4.

enumerator ADC_DIGI_IIR_FILTER_COEFF_8

The filter coefficient is 8.

enumerator ADC_DIGI_IIR_FILTER_COEFF_16

The filter coefficient is 16.

enumerator **ADC_DIGI_IIR_FILTER_COEFF_64**

The filter coefficient is 64.

Header File

- [components/esp_adc/include/esp_adc/adc_oneshot.h](#)

Functions

```
esp_err_t adc_oneshot_new_unit(const adc_oneshot_unit_init_cfg_t *init_config,  
adc_oneshot_unit_handle_t *ret_unit)
```

Create a handle to a specific ADC unit.

Note

This API is thread-safe. For more details, see ADC programming guide

- Parameters:**
- **init_config** – [in] Driver initial configurations
 - **ret_unit** – [out] ADC unit handle

- Returns:**
- ESP_OK: On success
 - ESP_ERR_INVALID_ARG: Invalid arguments
 - ESP_ERR_NO_MEM: No memory
 - ESP_ERR_NOT_FOUND: The ADC peripheral to be claimed is already in use
 - ESP_FAIL: Clock source isn't initialised correctly

```
esp_err_t adc_oneshot_config_channel(adc_oneshot_unit_handle_t handle, adc_channel_t channel,  
const adc_oneshot_chan_cfg_t *config)
```

Set ADC oneshot mode required configurations.

Note

This API is thread-safe. For more details, see ADC programming guide

- Parameters:**
- **handle** – [in] ADC handle
 - **channel** – [in] ADC channel to be configured
 - **config** – [in] ADC configurations

- Returns:**
- ESP_OK: On success
 - ESP_ERR_INVALID_ARG: Invalid arguments

```
esp_err_t adc_oneshot_read(adc_oneshot_unit_handle_t handle, adc_channel_t chan, int *out_raw)
```

Get one ADC conversion raw result.

Note

This API is thread-safe. For more details, see ADC programming guide

Note

This API should NOT be called in an ISR context

- Parameters:**
- **handle** – [in] ADC handle
 - **chan** – [in] ADC channel
 - **out_raw** – [out] ADC conversion raw result

- Returns:**
- ESP_OK: On success
 - ESP_ERR_INVALID_ARG: Invalid arguments
 - ESP_ERR_TIMEOUT: Timeout, the ADC result is invalid

```
esp_err_t adc_oneshot_del_unit(adc_oneshot_unit_handle_t handle)
```

Delete the ADC unit handle.

Note

This API is thread-safe. For more details, see ADC programming guide

- Parameters:** **handle** – [in] ADC handle

- Returns:**
- ESP_OK: On success
 - ESP_ERR_INVALID_ARG: Invalid arguments
 - ESP_ERR_NOT_FOUND: The ADC peripheral to be disclaimed isn't in use

```
esp_err_t adc_oneshot_io_to_channel(int io_num, adc_unit_t *unit_id, adc_channel_t *channel)
```

Get ADC channel from the given GPIO number.

- Parameters:**
- **io_num** – [in] GPIO number
 - **unit_id** – [out] ADC unit

- **channel** – [out] ADC channel

Returns:

- ESP_OK: On success
- ESP_ERR_INVALID_ARG: Invalid argument
- ESP_ERR_NOT_FOUND: The IO is not a valid ADC pad

```
esp_err_t adc_oneshot_channel_to_io(adc_unit_t unit_id, adc_channel_t channel, int *io_num)
```

Get GPIO number from the given ADC channel.

Parameters:

- **unit_id** – [in] ADC unit
- **channel** – [in] ADC channel
- **io_num** – [out] GPIO number
- - - ESP_OK: On success
 - ESP_ERR_INVALID_ARG: Invalid argument

```
esp_err_t adc_oneshot_get_calibrated_result(adc_oneshot_unit_handle_t handle,
adc_cali_handle_t cali_handle, adc_channel_t chan, int *cali_result)
```

Convenience function to get ADC calibrated result.

This is an all-in-one function which does:

- oneshot read ADC raw result
- calibrate the raw result and convert it into calibrated result (in mV)

Parameters:

- **handle** – [in] ADC oneshot handle, you should call `adc_oneshot_new_unit()` to get this handle
- **cali_handle** – [in] ADC calibration handle, you should call `adc_cali_create_scheme_x()` in `adc_cali_scheme.h` to create a handle
- **chan** – [in] ADC channel
- **cali_result** – [out] Calibrated ADC result (in mV)

Returns:

- ESP_OK Other return errors from `adc_oneshot_read()` and `adc_cali_raw_to_voltage()`

Structures

```
struct adc_oneshot_unit_init_cfg_t
```

ADC oneshot driver initial configurations.

Public Members

```
adc_unit_t unit_id
```

ADC unit.

`adc_oneshot_clk_src_t clk_src`

Clock source.

`adc_ulp_mode_t ulp_mode`

ADC controlled by ULP, see `adc_ulp_mode_t`

struct `adc_oneshot_chan_cfg_t`

ADC channel configurations.

Public Members

`adc_atten_t atten`

ADC attenuation.

`adc_bitwidth_t bitwidth`

ADC conversion result bits.

Type Definitions

typedef struct `adc_oneshot_unit_ctx_t *adc_oneshot_unit_handle_t`

Type of ADC unit handle for oneshot mode.

[Provide feedback about this document](#)

[← Previous](#)

[Next →](#)

© Copyright 2016 - 2023, Espressif Systems (Shanghai) Co., Ltd.

Built with [Sphinx](#) using a [theme](#) based on [Read the Docs Sphinx Theme](#).

[Download PDF](#)